TM223

Automation Studio diagnostics

Requirements		
Training modules	TM210 - Working with Automation Studio TM213 - Automation Runtime	
Software	Automation Studio 4.6 Automation Runtime 4.61 and later	
Hardware	X20 controller and X20 I/O modules ETA210 or ETAL210 + ETAL690 www.br-automation.com/eta-system	

Table of contents

1 Introduction

Automation Studio contains a variety of tools. The tools help during development and later during diagnosis. With the diagnostics tools, the system behavior can be recorded in detail.
All in all, Automation Studio combines the tools for diagnostics, commissioning and control into one single program.

The diagnosis begins with selecting the right tool. This training module shows how the diagnostics tools can be used individually or in combination. Numerous tasks contribute to better understanding and provide examples for practical use.
Automation Runtime provides access to diagnostic information. For this reason, a lot of diagnostic data is available directly in the System Diagnostics Manager. This can be launched from a web browser. Data is also available in the control application using library functions.

Figure 1: Automation Studio diagnostics

The diagnostics tools for integrated motion control are described in the training module "TM410 – Working with Integrated Motion Control". The diagnostics tools for integrated safety are dealt with in the training module "TM510 - Working with SafeDESIGNER". In addition, the training module "TM920 - Diagnosis and service" deals intensively with diagnosis and error correction.

1.1 Learning objectives

This training module uses selected examples illustrating different diagnostic possibilities during programming, commissioning and servicing to help participants learn how to work with the various diagnostics tools.

- Participants will learn the criteria for selecting the correct diagnostic tool.
- Participants will learn how to analyze and log general system information.
- Participants will learn how to monitor and record process values.
- Participants will get to know the options for system and application diagnostics.
- Participants will learn which Automation Runtime configuration options are relevant to which diagnostics tools.

1.2 Symbols and safety notices

Unless otherwise specified, the symbol descriptions and safety notices listed in "TM210 – Working with Automation Studio" apply.

2 The correct diagnostic tool

Selecting the correct diagnostic tool makes it possible to quickly and effectively localize the problem.
If irrelevant data is analyzed during the problem search or passed on to third parties, this leads to a considerable delay in solving the problem.

The Logger can be used to recognize a cycle time violation. However, the Logger does not display the cause of the cycle time violation.

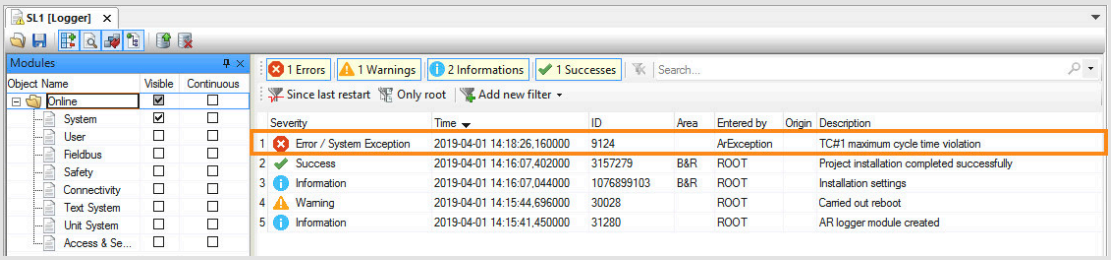


Figure 2: Cycle time violation in the Logger window

The cause of the error in the Logger will be given as a cycle time violation in Task Class #1. The backtrace data refers to the task where the cycle time violation occurred.

Situation 1

In the multitasking system, a task is interrupted by a higher priority task. This can be the cause of the cycle time violation. If the higher priority task has a longer execution time, the task entered in the Logger is no longer able to finish its configured cycle time + tolerance.

Situation 2

Several tasks are executed one after another cyclically in the same task class. If one of the previous tasks takes longer to complete, then the task shown in the Logger will likewise not be the cause of the cycle time violation.

Solution

In either case, the Logger alone does not help in determining the cause of the error. The problem can only be investigated in detail with the Profiler, see 5.1 "Perform runtime measurements in the Profiler" on page 26. The Profiler graphically displays the time sequence of the execution time of individual tasks.

2.1 Debugging procedure

There are a number of different ways to analyze a problem. Combining different localization and analysis strategies considerably increases effectiveness when trying to locate errors.

The following strategies are available:

- Analysis of environment and framework conditions
- Description of the error type
- Error isolation
- Measuring and recording data

Environment and general conditions

The environment of the machine and the general conditions under which it is operated are analyzed. This allows the error to be quickly localized.

Examples of framework conditions:

- Shift and batch change
- Indoor climate
- Replacement of sensors
- Operator actions
- Operating equipment

If all error sources, such as sensors and actuators, have been excluded in the machine environment, the analysis can be started in the control system and in the application software.

One-time or recurring errors

If errors are reproducible in certain actions, they can be investigated in a targeted manner.
Errors that do not occur with certain actions or that do not occur regularly are extremely difficult to find. In these cases the error must be reproduced in order to be able to investigate it. The analysis of sporadic errors is simplified by preparing the automation software.

Program or operation errors

Runtime errors are avoided by taking some conditions into account during the process flow.

Causes of runtime errors:

- Division by zero
- Missing evaluation of return values from functions
- Overflow when accessing array elements (e.g. loop counters)
- Access to non-initialized pointer

What is the next step?
The findings of the analysis are documented. Checklists help to record the data and provide support so that important information is not lost.

2.2 Checklists

Checklists do not just help when trying to analyze a problem during servicing; they are also very useful during configuration. Persons involved in troubleshooting enter the collected information in the checklists and can thus provide prompt support.

Checklist for describing the type of problem

- How are the surroundings and environmental conditions?
- What topology is used?
- Can the problem be reproduced, or did it occur only once?
- What actions need to be taken to reproduce the problem?
- When did the problem first occur?
- Have there been any changes in the software and/or hardware configuration or machine environment since then?
- What is the status of the controller, and what is the LED status of the installed components?
- What information was loaded onto the controller?
  e.g. Logger files, Profiler data, system dump

What should I bear in mind when filling out the checklist?
The more detailed the actions taken have been documented and information collected, the better the chances that the problem can be clearly identified and remedied. Further information about the error search in the control environment can be found in the training module "TM 920 - Diagnostics and Service".

Checklist for relaying information

Checklist: Software versions (also include any installed upgrades)

Software	Version	Description, remark
• Automation Studio	•	•
• Automation Runtime	•	•
• mapp Services	•	•
• [1]	•	•

Table 1: Checklist: Software versions

1 Automation Studio, Automation Runtime, mapp Technology, Safety Release, Visual components, etc.

Hardware used (also include installed hardware and firmware version)

Order number	Serial number Revision Hardware/Firmware	Description, remark
•	•	•
•	•	•

Table 2: Checklist used hardware

2.3 Overview of diagnostics tools

Automation Studio provides appropriate tools that can handle diagnostics during programming, commissioning and servicing.
Only by selecting the right diagnostic tool is it possible to accurately and quickly access the necessary information.
Automation Help is a constant companion when working with Automation Studio and provides detailed information about the various diagnostics tools.

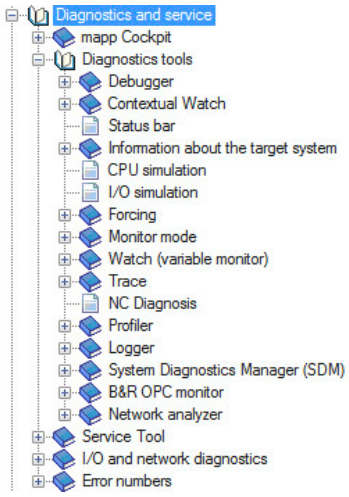


Figure 3: Category "Diagnostics and service" in Automation Help

Exercise: Open Automation Help chapter

The diagnostics tools are covered in section "Diagnostics and service". Get an overview of the structure of Automation Help in this area.

Diagnostics and service

Requirements for completing exercises in this training module

The descriptions and images in this chapter refer to the project designed in training module TM210 (Working with Automation Studio) or TM213 (Automation Runtime). The following exercises can be done with any Automation Studio project.

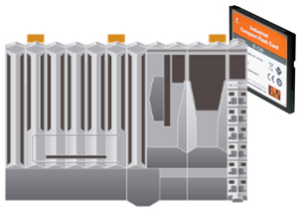


Figure 4: X20 controller with CompactFlash

Does the Profiler also work on the Automation Runtime Simulation?
Recording data with the Profiler and the Contextual Watch must be carried out on real hardware because no diagnostically conclusive measurement results arise in the Automation Runtime Simulation (ARsim).

3 Reading system information

System information can be read from the target system in Automation Studio and in the Web browser.

3.1.1 "Status bar" on page 8	Information about the status of the connection, Automation Runtime version and the operating state of the controller
3.1.2 "Online info dialog box" on page 8	Display of memory information, battery status and date/time configuration options for the controller
3.2 "Comparative functions " on page 9	Possibility of comparison between controller and project; software versions, connected modules, configuration files
3.3 "Error analysis in Logger" on page 12	Displays events that occur on the target system at runtime.
6.1 "System Diagnostics Manager (SDM)" on page 45	System Diagnostics Manager (SDM) is integrated directly into Automation Runtime. A web browser is used to read important information from the target system
3.4 "Hardware Configuration Analyzer" on page 16	The Hardware Configuration Analyzer calculates the cycle times of the individual POWERLINK and X2X networks and is used to control the system configuration

Table 3: Reading system information

3.1 Controller operating status

A number of options are available in Automation Studio for evaluating the operating status of a controller:

- 3.1.1 "Status bar" on page 8
- 3.1.2 "Online info dialog box" on page 8
- 6.1 "System Diagnostics Manager (SDM)" on page 45

3.1.1 Status bar

The status bar is located at the bottom of the Automation Studio window.

Information in the status bar

1. Connection settings
2. CPU type and Automation Runtime version
3. Current operating mode of the controller

Figure 5: The status bar

The status bar also displays license violations on the target system. Additional information can be found in Automation Help.

Project management \ The workspace \ Status bar
Real-time operating system \ Method of operation \ Operating states

3.1.2 Online info dialog box

If the online connection is active, information about the target system can be queried on the main menu under "Online" \ "Info...".

The online info dialog box contains the following elements:

- System type and Automation Runtime version
- Status of the internal battery backup
- Hardware node number
- Free memory
- Date and time of the target system.

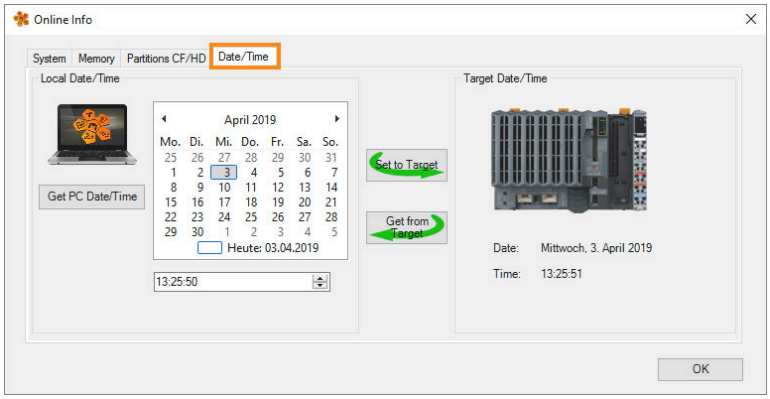


Figure 6: Setting the date and time on the target system

What does the controller need the date and time for?
In order to get diagnostically conclusive data, it is necessary for the time and date on the controller to be correct. Date and time are set in the online info dialog box or with library functions. In addition, the controller has a configuration option for time synchronization with a time server.

Diagnostics and service \ Diagnostics tools \ Information about the target system

Exercise: Set the time and date

In order to get diagnostically conclusive data, it is necessary for the time and date on the controller to be correct.

1) Open online info dialog box
2) Set target system time to PC time
3) Check the Logger entry in the "System" category, see 3.3 "Error analysis in Logger" on page 12.

3.2 Comparative functions

Automation Studio provides a diagnostic tool that compares the application with the target system.
The comparison function is used to check whether the hardware and software configuration of the opened project matches that of the target system. Differences are signaled using different text colors. Parameter values can then be applied.

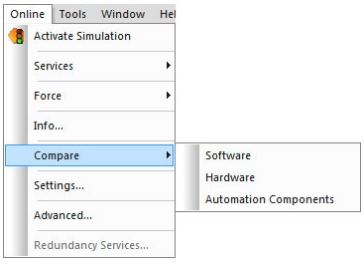


Figure 7: "Online" \ "Comparison"

3.2.1 Online software comparison

The online software comparison is used to compare the status and versions of tasks on the target system and compare them with the software configuration in the project. The online software comparison is opened by selecting "Online" \ "Compare" \ "Software" from the main menu.

The following information is analyzed:

- Comparison of software objects contained in the project with those on the target system
- Target memory of software objects
- Operating state of individual tasks
- Versions and timestamp of the last compile

Left: Projected elements of the software configuration
Right: Active software configuration on the target system

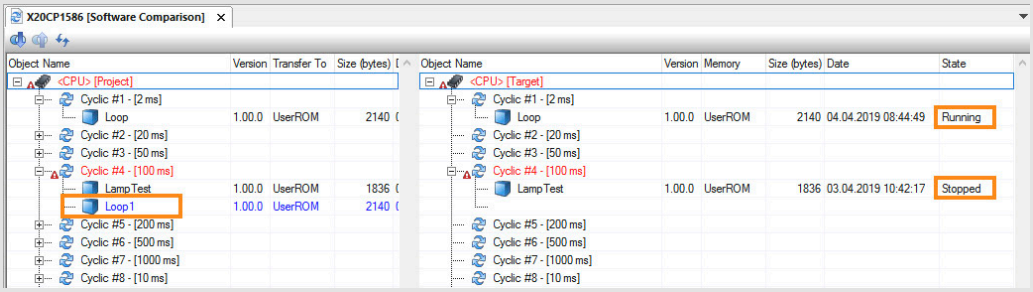


Figure 8: Online software comparison

In the example, the "LampTest" task on the target system has been stopped, whereas the "Loop1" task is not yet present on the target system.

With the online software comparison, configuration differences between the project and the target system can be recognized. Other tools are needed to compare the source code.

Further information:

- 5.4 "Source file comparison" on page 42
- 5.5 "Source file comparison on the target system" on page 43

Diagnostics and service \ Diagnostics tools \ Monitor mode \ Online software comparison

Software differences during online installation

During online installation, the differences between the software configuration in the project and the target system are checked.
The differences are displayed by clicking on the blue symbol in the Transfer dialog box.

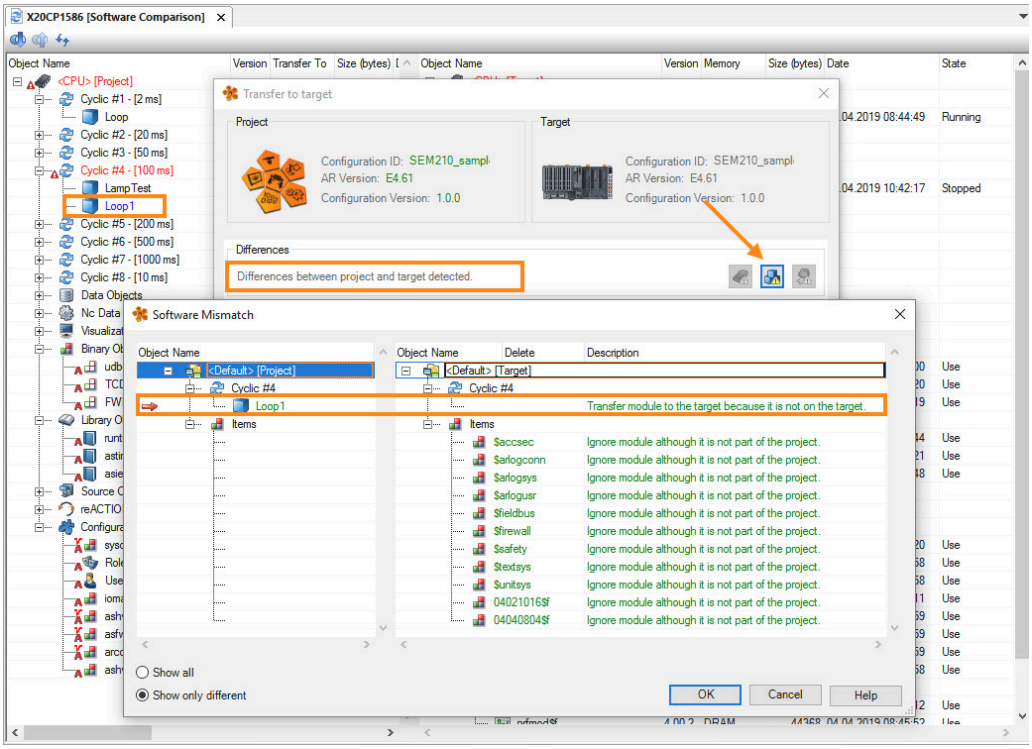


Figure 9: Display of software differences in the Transfer dialog box

Project management \ Project installation \ Execute project installation \ Transfer to target \ Differences dialog box

3.2.2 Online hardware comparison

The online hardware comparison compares the hardware configured in the project with the actual hardware identified at runtime. The online hardware comparison is opened by selecting "Online" \ "Compare" \ "Hardware" from the main menu.
Left: Projected elements of the hardware configuration
Right: Hardware configuration identified on the target system
Conflicts between the project and the target system are highlighted in red. Additional entries on the project or target system page are highlighted in green.

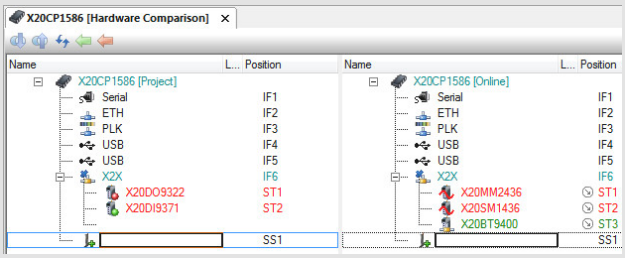


Figure 10: Online hardware comparison

In the example, two digital modules were configured on the X2X Link. However, two other modules and one additional module were identified on the target system.

Diagnostics and service \ Diagnostics tools \ Monitor mode \ Online hardware comparison

3.2.3 Online comparison of automation components

The online comparison of Automation Components compares the configuration objects configured in the project with the actual configuration objects identified at runtime. In the overview, differences at object level are compared in detail with configuration objects.
The online comparison of the Automation Components can be opened from the main menu under "Open" \ "Comparison" \ "Automation components".
Left: Configured configuration objects
Right: Configuration objects loaded on the target system

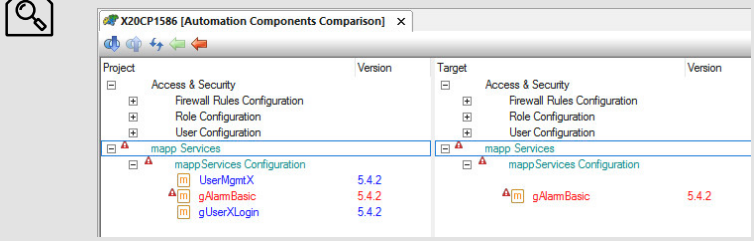


Figure 11: Online comparison of automation components

In the example, a change was made to the project in "gAlarmBasic" and not yet transferred to the target system. "UserMgmtX" and "gUserXLogin" were added to the project, but not yet transferred to the target system.

Applying parameter values

The detailed comparison shows the values of the configuration object in the Automation Studio project alongside the current values on the controller. The values for selected parameters are applied to the Automation Studio project by selecting a parameter or an entire parameter group (1) and then uploading via the toolbar (2).

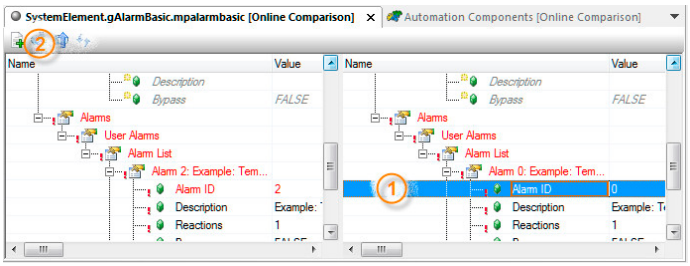


Figure 12: Applying parameter values in the detailed comparison

Diagnostics and service \ Diagnostics tools \ Monitor mode \ Online comparison of automation components

- Overview comparison
- Detailed comparison

3.3 Error analysis in Logger

Automation Runtime logs all fatal errors (e.g. cycle time violations), warnings and information messages (e.g. warm restarts) that take place when the application is executed.
This log is stored in the controller's memory and is available after restarting.

Diagnostics and service \ Diagnostics tools \ Logger

- Opening the Logger window
- User interface description \ Backtrace
- Operating the Logger \ Storing / Loading Logger data

3.3.1 Logger with an active online connection

The Logger is opened from the menu in the Physical View under "Open" \ "Logger" or by using the key combination **<CTRL> + <L>**.

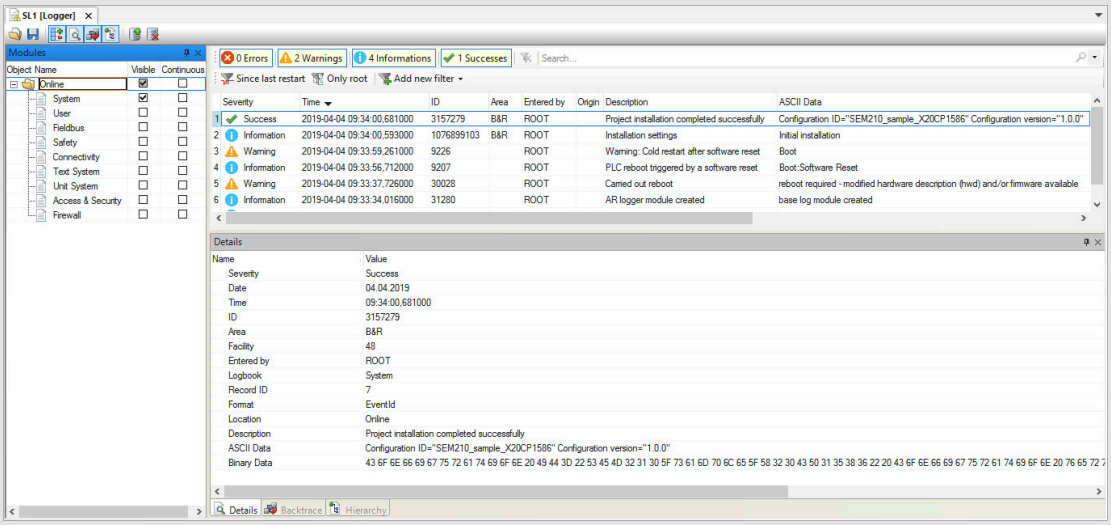


Figure 13: Logger window

Events logged by Automation Runtime are shown here. After creating the offline installation, the controller started up successfully.

Exercise: Cause a cycle time violation and check Logger entries

By incrementally increasing the variable "udEndValue" in the variable monitor in the program "Loop", a cycle time violation is achieved.
Once an online connection has been reestablished between Automation Studio and the target system after restarting, open the Logger window and look for the cause of the boot into the "SERVICE" operating mode.

1) Open the Watch window of the "Loop" task, see 4.1 "Monitoring and modifying variables" on page 18
2) Increase the "udEndValue" variable little by little until the cycle time violation occurs; this is followed by a connection abort and startup in the "SERVICE" mode
3) Open Logger
4) Search for the cause of operating mode "SERVICE"
5) Select the corresponding Logger entry and press the **<F1 key>**

Once opened, the Logger indicates the cause of booting in the SERVICE operating mode. The program that is causing the problem can be quickly identified with backtrace in this case.

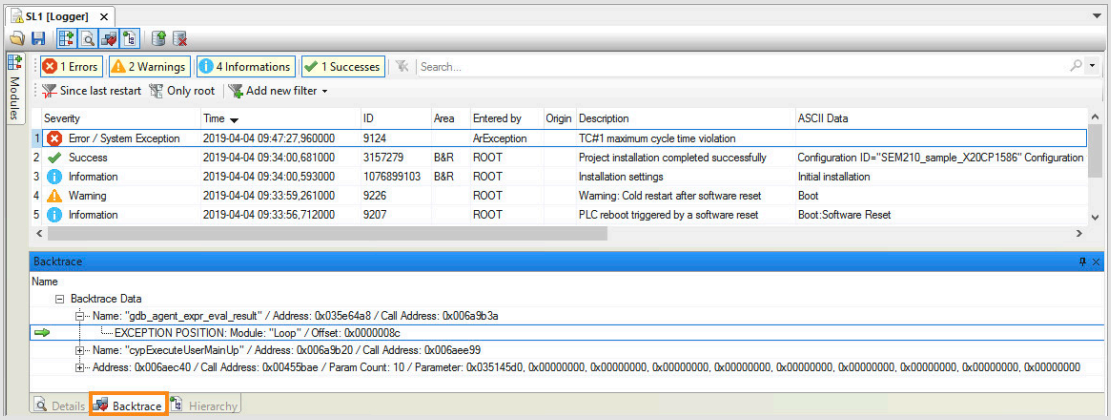


Figure 14: Cycle time violation in the Logger

To get a detailed error description of the Logger entry, select the entry and press the **<F1 key>**. Automation Help automatically displays the appropriate text. Additional information about error entry is available by displaying the backtrace.

This opens a description of the selected error number in Automation Help.

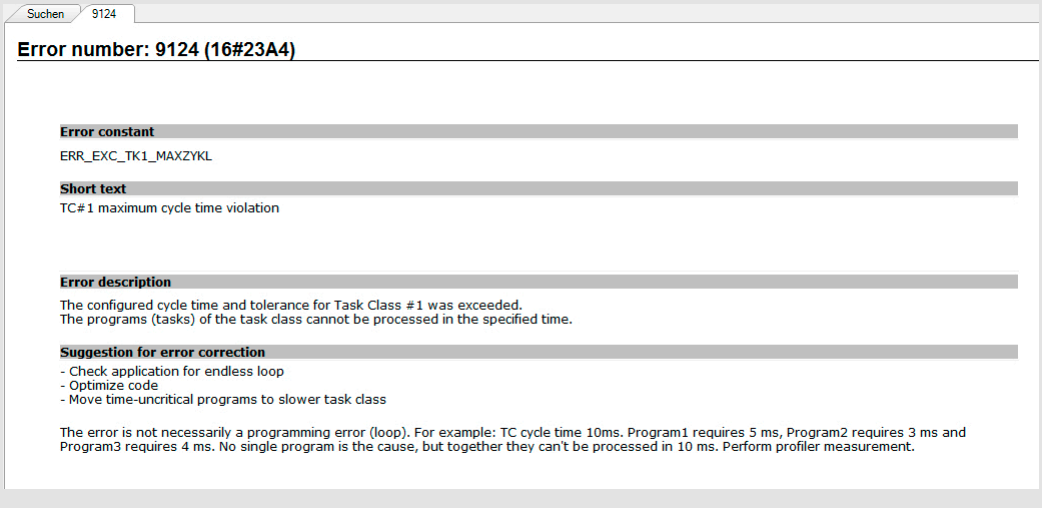


Figure 15: Context-sensitive help for Automation Runtime errors

3.3.2 Offline evaluation of Logger data

The recordings of the Logger can also be evaluated without a connection to the controller.
Nonetheless, the data itself must always be uploaded by means of an existing online connection to Automation Studio or with the aid of System Diagnostics Manager System Diagnostics Manager (SDM).

System dump use case

The Logger data is saved by the service engineer individually using System Diagnostics Manager or as a system dump. The transferred data can now be opened in Automation Studio and analyzed.
The Logger entries can be saved and loaded in the toolbar of the Logger in Automation Studio.

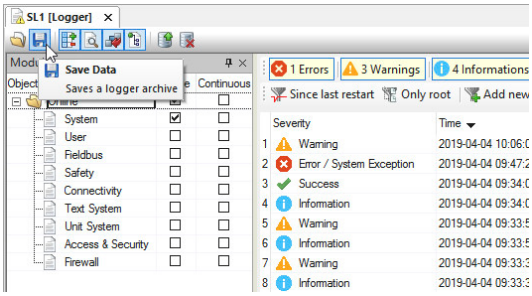


Figure 16: Saving Logger entries

3.3.3 Generating user log data

Logger functions can also be used by the application program to log certain events.
This is handled using the functions in the **ArEventLog** library. The functions contained in this library allow 32-bit event IDs to be entered. They are distinct within the system. Errors, warnings, information and successes are differentiated. Event IDs from the user and from the system can be easily differentiated.
The user cannot enter any event IDs used by the system in the Logger. This is prevented by the library.

The 32-bit event IDs are put together as follows:

Bits 31-30	Bit 29	Bit 28	Bits 27-16	Bits 15-0
Severity	1 .. Customer	Reserve	Facility	Code

User event IDs are generated according to the represented circuit diagram or by using the ArEventLogMakeEventID() function.
This facility makes 12 bits available to clearly identify ranges. For user event IDs, the facility is divided up as follows:

- Values 0 to 15: Customer applications
- Values 16 to 4096: Range for device manufacturers and special cases

Programming \ Libraries \ Configuration, system information, runtime control \ ArEventLog

Use cases:

- Logging service actions, e.g. battery replaced
- Logging user actions, e.g. forbidden entries
- Retrieving events of an exception task and entering them in the Logger
- Logging events when module monitoring is disabled, e.g. ModulOK = FALSE

Exercise: Create a user logger entry

Create a user logbook in the existing Automation Studio project. The warning "**This is a warning entry from the user**" is entered in this Logbook (Severity = Warning).
For this purpose, the library example of the library "ArEventLog" is used. After importing and transferring the sample program, the individual functions are enabled via the Watch window.

1) Click on the project and filter for "Examples" in the toolbox
2) Select "Libraries Examples" and import to Samples \ Library \ ArEventLog \ "LibArEventLog_ST.zip"
3) Performing project installation
4) Show variable "EventLog" in the Watch window
5) Create the user logbook via ""Commands" \ "CreateUserLog"
6) Generate a 32-bit event ID via Automation Help

Programming \ Libraries \ Configuration, System info, Runtime control \ ArEventLog \ Function blocks and functions \ 32-bit event ID

7) Enter the number via "Event" \ "EventID"

8) Right click on "AdditionalData", click on the string and enter the text

9) Create the user Logger entry via Commands" \ "WriteUserEvent"

10) Open Logger and check the results

After creating the user logbook "UsrEvLog" and the user Logger entry, the warning is visible in the Logger.

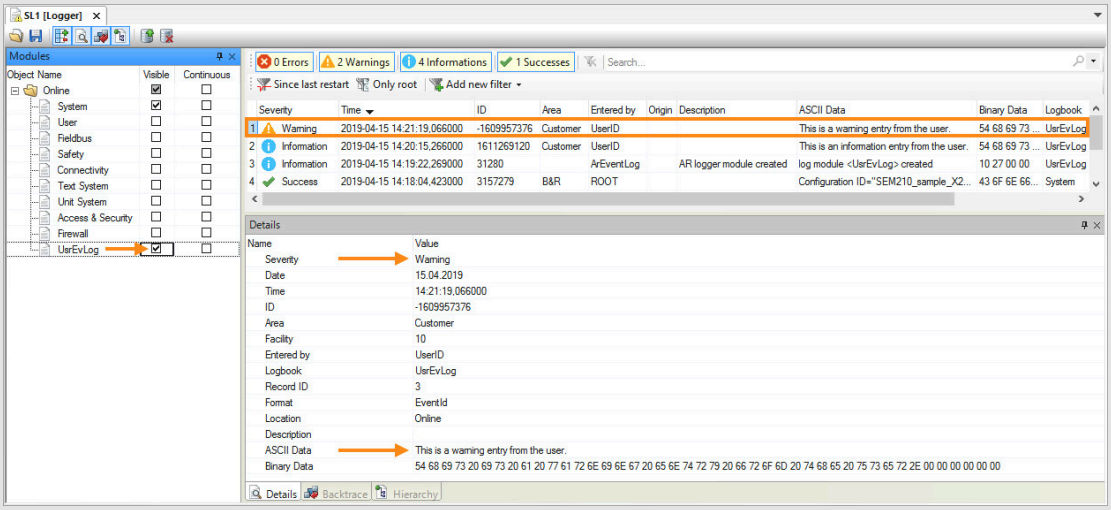


Figure 17: The user Logger entry (warning) was generated by the library "ArEventLog"

Programming \ Examples \ Adding examples
Programming \ Examples \ Examples - Libraries \ Configuration, system info, runtime control \ User event handling with ArEventLog

3.4 Hardware Configuration Analyzer

The Hardware Configuration Analyzer calculates cycle times for POWERLINK and X2X networks. It is verified whether it is possible to adhere to the set cycle times with the configured hardware configuration. To this end, the calculated run times are compared with the configured cycle times. Misconfigurations can be detected. The number of hub levels as well as the configured cable lengths are also included in the calculation.
The Hardware Configuration Analyzer is opened via the main menu "Open" \ "Hardware Configuration Analyzer".

A POWERLINK bus coupler was inserted in the hardware configuration. The X2X interface of the bus controller was configured at a cycle time of 200µs. The Hardware Configuration Analyzer shows that the calculated runtime is 230µs and therefore this configuration is faulty.

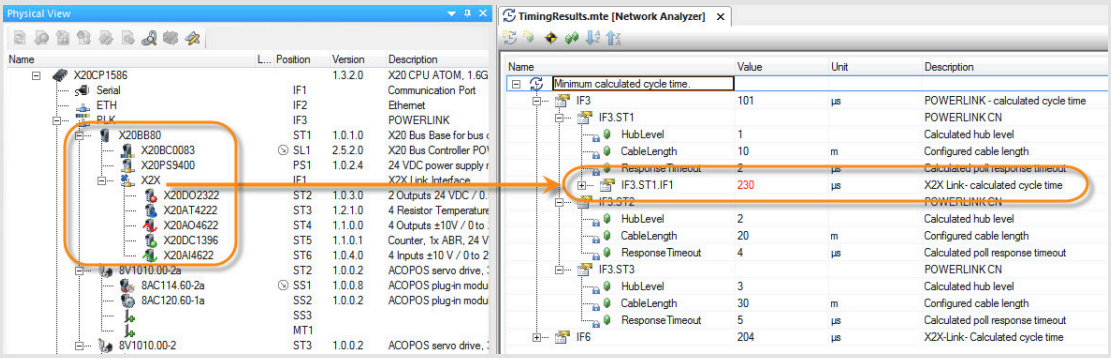


Figure 18: Viewing the data as a table

A summary of the results of the Hardware Configuration Analyzer is also shown in the output window. The "Position" column also shows which configuration entry must be checked and corrected.

#	Category	Date/Time	Description	Position
20	Error	24.04.2017 10:20:57,8500	Analyze networks finished. 1 cycle time violation detected. For detailed results see the entries below.	
21	Info	24.04.2017 10:20:57,8812	X20CP1586.IF3: Calculated POWERLINK cycle time (101 µs) is lower or equal than configured time (2000 µs).	Module: X20CP1586.IF3, Property: CycleTime
22	Error	24.04.2017 10:20:57,8812	X20BB80.IF1: Calculated X2X Link cycle time (230 µs) is higher than configured time (200 µs).	Module: X20BB80.IF1, Property: CycleTime

Figure 19: Cycle time violation in X2X cycle time in the output window

In addition to the tabular representation of the calculated results as well as the summary in the output window, configuration problems are marked with a red triangle symbol in the System Designer.

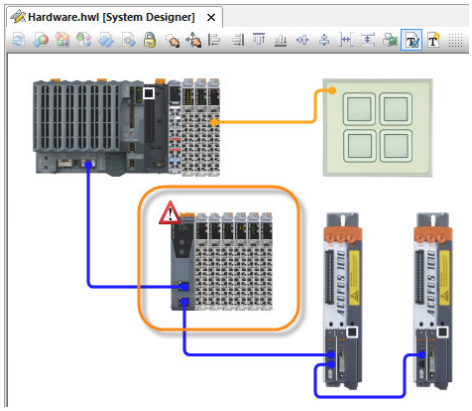


Figure 20: Display of a configuration problem in System Designer

Diagnosis and service \ Diagnostic tool \ Hardware Configuration Analyzer
Diagnostics and service \ I/O and network diagnostics

Exercise: Check the cycle time configuration with the Hardware Configuration Analyzer

It is useful to make statements about the entire system load at any time. The cycle time configuration for X2X and POWERLINK networks should now be checked. Set the calculated load for the X2X and POWERLINK interfaces configured in the project using the Hardware Configuration Analyzer.

1) Open the Hardware Configuration Analyzer

2) Analyzing entries in the output window and in the tabular view

3) Compare calculated cycle time with configured cycle time

4 Monitoring and analyzing process values

Process values can be monitored, analyzed and modified in many different ways in Automation Studio.

4.1 "Monitoring and modifying variables" on page 18	The Watch window allows variable values to be monitored and modified.
4.2 "Recording variables in real time" on page 20	You can use a Trace to record multiple values over a specific time domain in real time. Automation Studio can load the data and display it as a curve.
4.3 "I/O monitor and forcing" on page 24	The I/O monitor makes it possible to read the values of I/O variables and status information of I/O modules.

Table 4: Monitoring and analyzing process values

Requirements for the exercises of this section

First, a small program will be added to the existing project. The program contains all necessary process variables to perform the exercises in this section.

Exercise: Create the "signal" program

For the other exercises, process variables that can be changed are required. For this purpose, a program is added in the Structured Text programming language. Then the variables are implemented and declared.

1) Add a new Structured Text program in Logical View
2) Rename program to "signal"
3) Assign program to task class #1
4) Implement the program function:

   If a digital input is set to TRUE, a number is incremented in each cycle. The number has the data type USINT.
   At the end of the program, a total sum is increased in each cycle by the value of the number. The data type of the total sum is INT.

```
PROGRAM _CYCLIC
    (*add 1 each cylce when input is TRUE*)
    IF diEnableRamp = TRUE THEN
        rampSignal := rampSignal + 1;
    ELSE
        rampSignal := 0;
    END_IF

    (*add value to the sum each cycle*)
    sumOverTime := sumOverTime + rampSignal;
END_PROGRAM
```

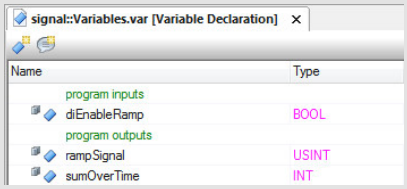


Figure 21: Variable declaration for the "signal" program

4.1 Monitoring and modifying variables

The Watch window allows the values of variables on the target system to be displayed, monitored and modified. Variable lists are saved in the Watch window for diagnostic and function tests with use and are reused at a later time.

Exercise: Operate and diagnose the "signal" program

The previously created "signal" program should be operated and monitored using the Watch window in Automation Studio.

First, the Watch window is opened via the shortcut menu for the "signal" program. Then all available process variables are added to the Watch window.

Overwriting of process variables during operation is only permitted for authorized personnel.

4.1.1 Adding process variables to the Watch window

It is necessary to check if the current project status has been transferred to the controller. This can be done, for example, via the online software configuration. Then the variable monitor is opened and the process variables are added and monitored.

Step 1

The Trace dialog box is opened in the software configuration using the shortcut menu for the corresponding task "Open" \ "Watch".

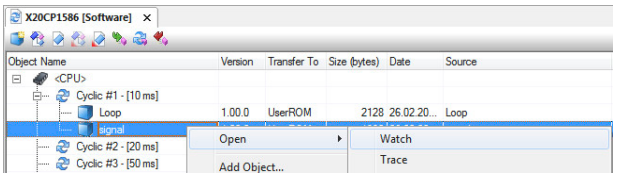


Figure 22: Opening the Watch window

Step 2

The variables can be added via the "Insert variable" button or the **<Insert key>**.

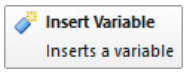


Figure 23: Adding variables

Once the variables have been added in the Watch window, the process sequence of the application can be simulated.

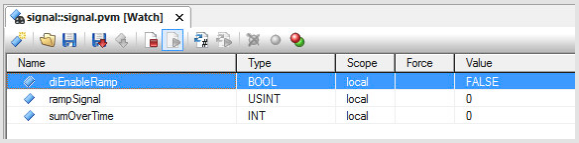


Figure 24: Display the variables in the Watch window

4.1.2 Operating the "signal" program in the Watch window

Variables on the controller can be monitored and modified using the Watch window. The data type, scope and so on are displayed next to the value of the variable.

Step 1

Start the process
The process is started by setting the value of the variable "diEnableRamp" to TRUE.

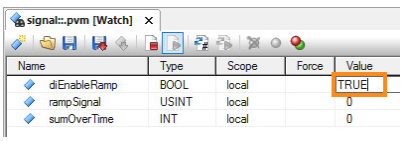


Figure 25: Set variable

Step 2

Check the status of the process
Variable "rampSignal" counts up to overflow and then starts again with the value 0.

Step 3

Stop the process
The value of the variable "rampSignal" is 0; the value of "sumOverTime" remains at the last calculated value.

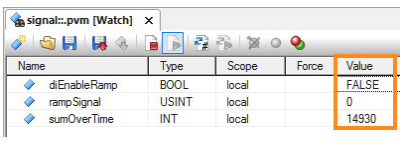


Figure 26: Process stopped

Saving the variable list

The variable list in the variable monitor should be saved for later use. This way the used variable list can be restored at any time.

Step 1	Step 2
Save the variable list using the "Save data" icon.	Enter the name of the variable list. Several lists can be managed depending on the application.

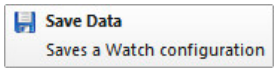


Figure 27: Saving the variable list

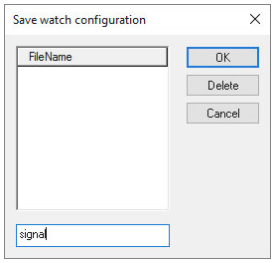


Figure 28: Name of the variable list

Diagnostics and service \ Diagnostics tools \ Watch window

4.1.3 Changing multiple values simultaneously in the Watch window

If a value is changed in the Watch window, it will be transferred to the controller immediately after the **<ENTER key>** is pressed. The controller will then apply the new value in the next cycle.
To enter several values in the Watch window without immediately transferring data to the controller, proceed as follows.

Step 1	Step 2
Enable the archive mode using the "Archive mode" button.	Enter the values to be changed. Transfer the values to the controller using the "Write values" button.

Figure 29: Enable archive mode

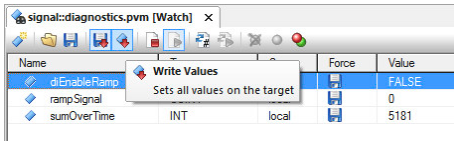


Figure 30: Changing and transferring values

Diagnostics and service \ Diagnostics tools \ Watch window \ Archive mode

4.2 Recording variables in real time

In the Watch window, Automation Studio reads the variable values asynchronously from the controller.
However, this type of asynchronous accessing of the actual value changes in the Automation Runtime task class system leads to the following limitations:

- Value displayed asynchronously to the task class
- Unable to determine series of value changes and their dependencies

The "**Trace**" function can be used to record changes in values on the target system in real time and synchronous to the task class.
By analyzing trace data, processes in the application can be optimized and errors detected.

The following example shows how another process is started when the state of a particular variable is changed. The measurement cursor can be used to establish the time difference between the corresponding value changes of both curves.

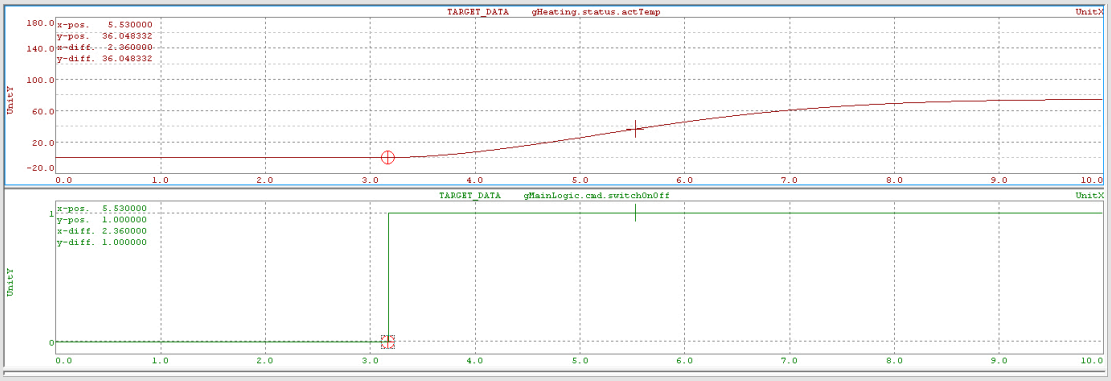


Figure 31: Example of a trace recording

Exercise: Record a curve that depends on other variables

When running the "signal" program, the variables are interdependent. When the process is activated by setting variable "diEnableRamp" to TRUE, all other variables are affected. The dependencies are recorded using the Trace.

1) 4.2.1 "Opening the Trace window and adding variables" on page 21
2) Configure trigger condition "diEnableRamp", see 4.2.2 "Editing the Trace configuration." on page 22
3) 4.2.3 "Starting the process and evaluating trace data" on page 23

4.2.1 Opening the Trace window and adding variables

Step 1

The Trace dialog box is opened in the software configuration using the shortcut menu for the corresponding task "Open" \ "Trace".

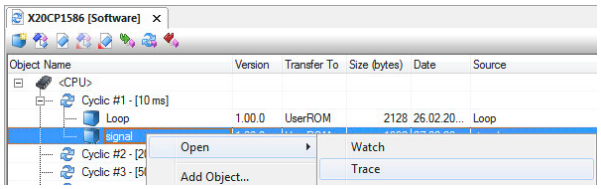


Figure 32: Open the Trace dialog box from the software configuration

Step 2

Create a new trace configuration using the "Insert trace configuration" button.

Figure 33: Add Trace configuration

Step 3

Add the variables to be recorded with the "Insert new variable" button.
In this example, add all available variables from the "signal" program.

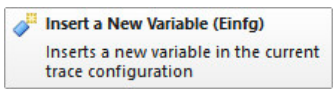


Figure 34: Adding variables to the Trace configuration

Step 4

Install the trace configuration on the target system using the "Install" button.

Figure 35: Install trace configuration on the target system

The trace configuration now looks like this:

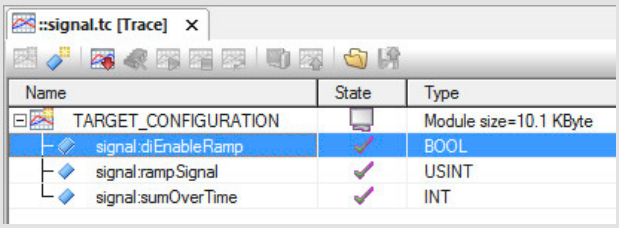


Figure 36: Completed trace configuration

Values are recorded cyclically in the context of the task class. The period and start condition of the recording can be configured in the Trace configuration's properties.

4.2.2 Editing the Trace configuration.

The Trace configuration is not permitted to be on the target system if it is to be processed. If necessary, the Trace configuration must first be uninstalled by the target system.

Step 1

Open the Trace configuration via the "Properties" shortcut menu.

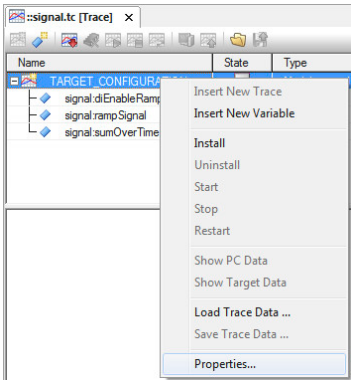


Figure 37: Trace properties

Step 2

In the "General" tab, adjust the recording buffer to the recording duration.
In this example, entries = 3000

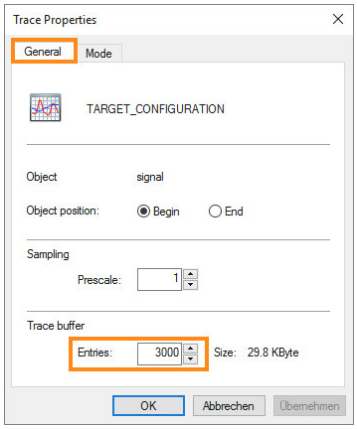


Figure 38: Edit Trace buffer

Step 3

In the "Mode" tab, select the option "Start Trace if trigger condition is TRUE".
Click the button "Trigger condition...".

Step 4

Formulate the trigger condition. Click "OK".
In this example "diEnableRamp" = 1.
Activate the checkbox "Only record if the trigger condition is true".

Step 3	Step 4

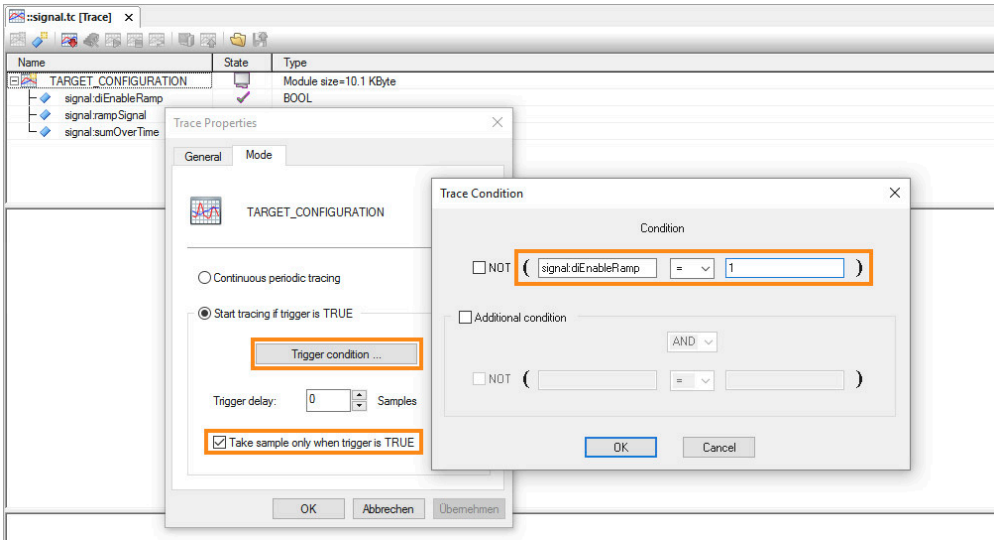


Figure 39: Define the trigger condition

How is the variable selected in the trigger condition?
The dialog box for selecting variables is opened by pressing the **<space bar>**.

The Trace configuration has been completed. This configuration must now be transferred to the target system. The configuration is transferred via the "Install" icon in the toolbar.
Recording now takes place automatically as soon as the trigger condition is met.

4.2.3 Starting the process and evaluating trace data

If the Trace configuration was transferred to the target system with the "Install" icon and the trigger condition is fulfilled, the trace data is recorded.

Step 1	Step 2
Open the Watch window. In the example, add all variables of the "signal" program.	Start the process. In the example, set the variable "diEnableRamp" to TRUE.

Step 3	Step 4
Stop the recording with the "Stop" button. 	Display the trace data with the button "Display trace data of the target system". 

Data is recorded when the trigger condition has been met. Values can be modified as needed in the variable monitor. After uploading, the recorded variables are displayed as individual curves. The curves can also be superimposed (right-click "Overlay graphs"), resulting in the following view:

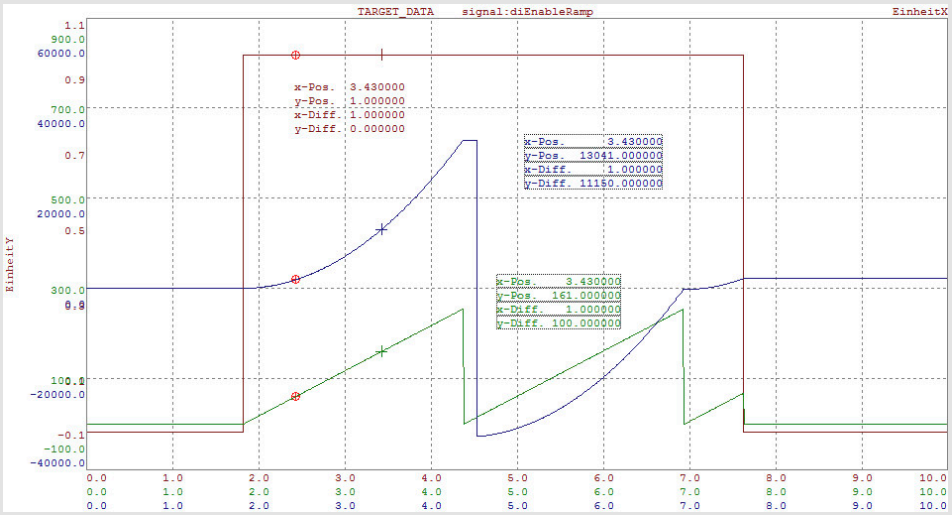


Figure 40: The graphs of 3 variables superimposed on each other

Using the measurement cursor, value changes and differences are determined exactly.

Diagnostics and service \ Diagnostics tools \ Trace window

4.3 I/O monitor and forcing

Double-clicking on a module in the Physical View opens the I/O mapping window. The physical status of the I/O channels is displayed when there is an active online connection and the active monitor mode is selected.

Figure 41: Switching on the monitor mode

Channel status and status data points

Via the I/O mapping, the channels of an I/O module are directly assigned to process variables. In addition to the channel values, further information is also available. The control application can be used to evaluate whether the module has been detected and configured at runtime, via the channel "ModuleOk". Additional status inputs allow a diagnosis of the I/O channels in the application.

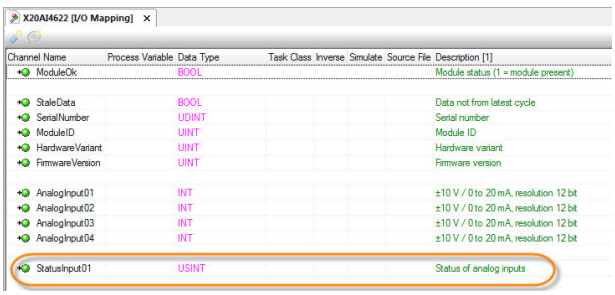


Figure 42: Module information in the I/O mapping

The channel "StatusInput01" is shown in the figure. This contains bit-by-bit information about the individual channels of the module. This makes it possible to detect short circuits, wire breaks and other conditions. A detailed breakdown of the diagnostic information is documented in the register description in the data sheet of the I/O module used. Depending on the type of module, further status information can be activated in the I/O configuration.

Forcing

The option "**Force**" makes it possible to assign any of the I/O data points a value, regardless of their actual physical value. The function "Force" is enabled in the I/O mapping and in the Watch window for variables that are linked to the I/O data points.

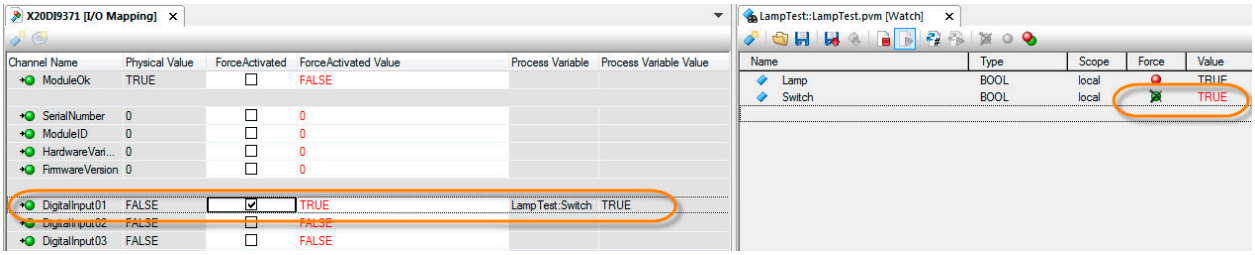


Figure 43: I/O mapping in monitor mode; forcing I/O channels in the I/O mapping and in the Watch window

When forcing inputs of an input module (e.g. X20DI9371), the user program operates with the "force" value rather than the actual input state.

When forcing outputs of an output module (e.g. X20DO9322) it is written directly to the output of the corresponding hardware. This is independent of the value calculated by the user program.

Before leaving a machine, it must be ensured that there are no force operations still in effect. This is disabled automatically by **restarting** the control system or using the menu item "Online" / "Force" / "Global force off".

Diagnostics and service \ Diagnostics tools \ Monitor Mode \ Mapping I/O channels in monitor mode
Diagnostics and service \ Diagnostics tools \ Force
Diagnostics and service \ I/O and network diagnostics

5 Software analysis during programming

There are several different diagnostics tools available in Automation Studio that provide support when designing the application software and for the error search at run time.

Software analysis during programming	
5.1 "Perform runtime measurements in the Profiler" on page 26	The Profiler can be used to measure and display important system data such as task runtimes, system and stack loads, etc.
5.2.3 "Line coverage" on page 35	Line coverage indicates the lines of the source code that are currently being executed.
5.2.4 "Contextual Watch" on page 36	Contextual watch is a tool for displaying the value of variables and parameters at exactly defined source code positions.
5.2.5 "Debugging the source code" on page 37	The debugger makes it easier to search for errors in the source code of a program or library.
5.2.7 "Evaluating event IDs, status variables and return values" on page 40	Status variables are used to identify the status when calling functions and function blocks in the user program
5.3 "Using variables in the programs" on page 40	The output window is used to display information about ongoing processes, e.g. building, downloading, generating the cross-reference list, displaying search results, etc.
5.4 "Source file comparison" on page 42	With the Automation Studio source file comparison, it is possible to compare individual files (programs, libraries, packages and data objects) or entire projects.

Table 5: Software analysis during programming

5.1 Perform runtime measurements in the Profiler

Automation Runtime continually records all runtime behavior. Using the Profiler, the runtimes of individual user tasks, the CPU load and different system events can be recorded. The Profiler is opened by selecting "Open" / "Profiler" from the main menu.
In the Profiler, different views for displaying the recorded data are offered. These are switched in the toolbar of the Profiler. A table view, a graphic view and a raw data view are available. The most important functions of the Profiler, such as opening the configuration, and starting and stopping the recording, are also controlled via the toolbar.

Figure 44: Toolbar in the Profiler

Requirements for the exercises of this section
For the following tasks, it's recommended to leave the Watch window open for the tasks "Loop" and "Loop1", the software configuration and the Profiler.

Diagnostics and service \ Diagnostics tools \ Profiler
- Recording Profiler data
- FAQ

B&R tutorials:

Automation Studio \ Diagnostics tools \ Profiler

5.1.1 Configuring the Profiler and carrying out recording

In order to get diagnostically conclusive measurement results, the recording duration and the event types must be configured. For the following tasks, only the execution times of the user tasks, task classes and exceptions are recorded. Short instructions follow, explaining how to edit the configuration and record data.

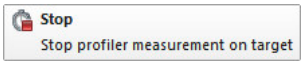
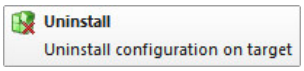
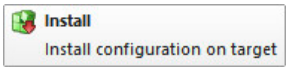
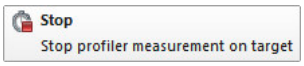
1	In order to edit the configuration, you must stop the current recording.	
2	The existing Profiler configuration is uninstalled from the target system.	
3	To open the dialog box for making configurations, click the "Configuration" icon in the Profiler toolbar.	
4	The changes are transfered to the target system using the "Install" icon in the toolbar. Recording begins again immediately with the new configuration.	
5	The current recording is stopped using the "Stop" icon.	
6	The Profiler data is loaded to and displayed in Automation Studio using the "upload data object" icon.	

Table 6: Overview of the required steps for changing the Profiler configuration and uploading the Profiler data

The configuration must be carried out according to the following figure:

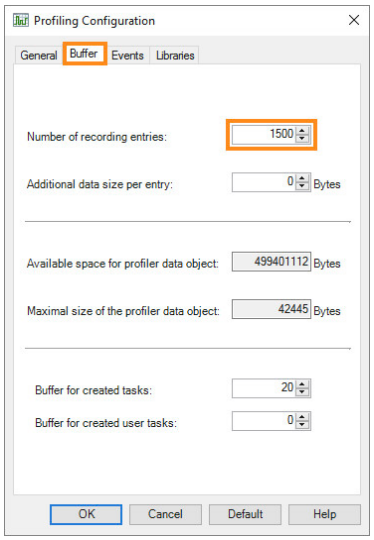


Figure 45: Profiler configuration: Recording buffer

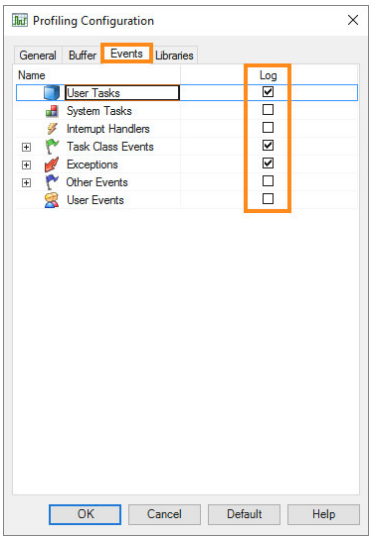


Figure 46: Profiler configuration: Event type

When are the checkboxes in the "Events" tab checked?
If the Profiler data is relayed to third parties, it is recommended to activate all the checkboxes in the "Events" tab. Filtering can be performed later.

Diagnostics and service \ Diagnostics tools \ Profiler \ Preparing the Profiler

Exercise: Measure the execution time of program "Loop"

In the task "Loop", which runs in task class #1, the value of the variable "udEndValue" is incremented stepwise in the Watch window. The resulting runtime behavior of the task and the available remaining times are monitored with the Profiler.

1) Set the "udEndValue" variable to 50000

2) Increase the "udEndValue" variable in steps

Results should be analyzed in the Profiler between each of these steps.

5.1.2 Profiler table and graphic views

Different views are available for analyzing the Profiler data. The data uploaded in the preceding task is now displayed in table or graphic form.

Table display of Profiler data

The table view of the Profiler data is opened using the "Table" icon.
With appropriate filtering, the execution times and CPU load are displayed for each task.

Name	CPU Usage [%]	Tolerance Count	Object Priority	Call Count	Minimal Net Time [µs]	Average Net Time [µs]	Maximal Net Time [µs]	Minimal Gross Time [µs]
Cyclic #1	2.145		230		217.356	218.539	219.832	965.138
loop	1.943		230	15	201.461	202.663	203.198	201.461
Cyclic #4	0.142		198		216.306	216.306	216.306	216.306
System Tasks	6.059							
Interrupt Ha...	1.061							
Idle Tasks	90.593							

Figure 47: Analyzing the CPU load with the Profiler

In order to get informative Profiler graphs, the call count of the slowest task class in the system must be >3. The recording duration of the Profiler is set via the buffer size in the Profiler configuration.

Diagnostics and service \ Diagnostics tools \ Profiler \ Recording Profiler data \ Analyzing Profiler data \ Analyzing Profiler data in table form

Visual display of Profiler data

The graphic view of the Profiler data is opened using the "Graphic" icon. The representation can be shown as in the following image.
Comprehensive analysis options are available via filter, zoom and measurement cursor functions.

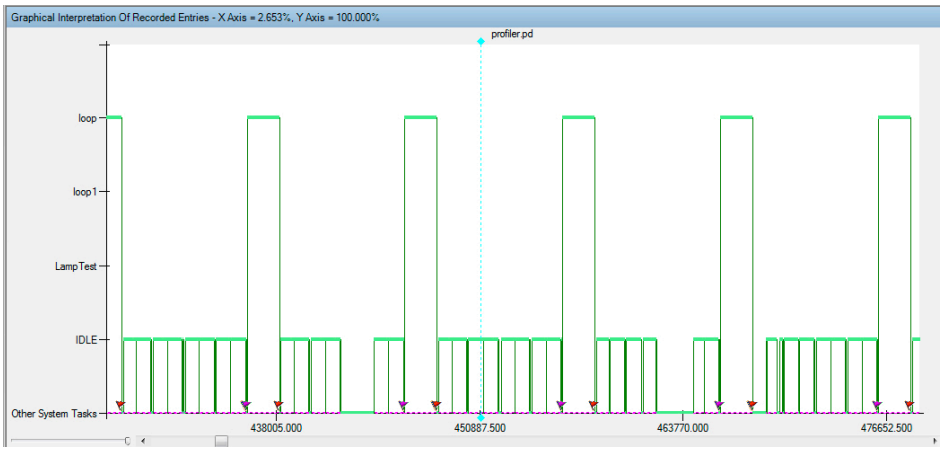


Figure 48: Result of the first Profiler measurement

Diagnostics and service \ Diagnostics tools \ Profiler \ Recording Profiler data \ Analyzing Profiler data \ Show Profiler data as graph

- Navigating in the graph view
- Zooming in the graph view

5.1.3 Measuring program runtime in the Profiler

The program runtime is measured with the measuring cursor in the Profiler toolbar. At the beginning, a reference cursor is set at the end of "Loop".

Exercise: Increase the "udEndValue" variable in steps

The Profiler is restarted in this step. Increase the value of the variable "udEndValue" in steps, e.g. to 10000. Use the Profiler to monitor the execution time of the "Loop" task. In addition, when a value is changed, the recording is stopped and uploaded.

The "Loop" task operates in a 10 millisecond task class. Looking more closely at the image, a cycle time violation must have occurred since the execution time has already reached 16.5 milliseconds.
The tolerance – defined in the properties of task class #1 as 10 milliseconds – now takes effect.

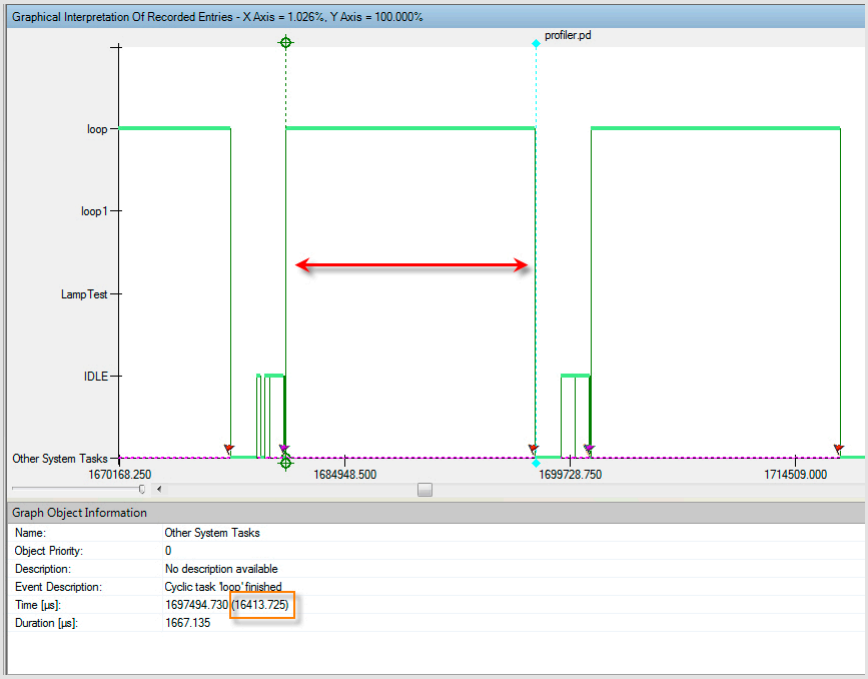


Figure 49: Exceeding the cycle time, effect of tolerance

With the value "Tolerance count", it can be determined in the table view if the tolerance time is already active in a task class. In the following image, the middle runtime of task class #1 is 10.7 ms. The tolerance count shows that the configured execution time was already exceeded 271 times.

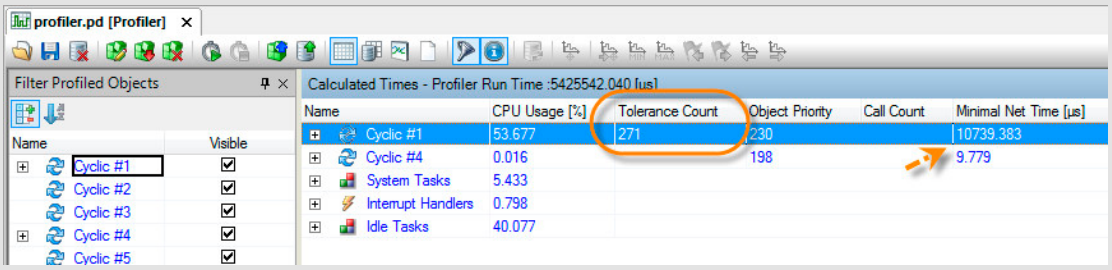


Figure 50: Parameter "Tolerance count" in the Profiler

The example shows the effect of using the tolerance on the average CPU load. Task class #1 causes a CPU load of around 54% even though the configured cycle time of 10 ms was exceeded. In this case, when the tolerance becomes effective, the task class #1 is only called again every 20 ms.

Exercise: Record task classes with different priorities

Task class priority makes it possible for a task in a higher priority task class to interrupt a task in a lower priority task class that takes longer.
Based on the tasks "Loop" and "Loop1", the value of the variable "udEndValue" is changed so that the task "Loop1" is interrupted exactly **two times** by the task "Loop". Set the starting value for the "udEndValue" variable in the "Loop" task to 2,000. A Profiler measurement could look as follows:

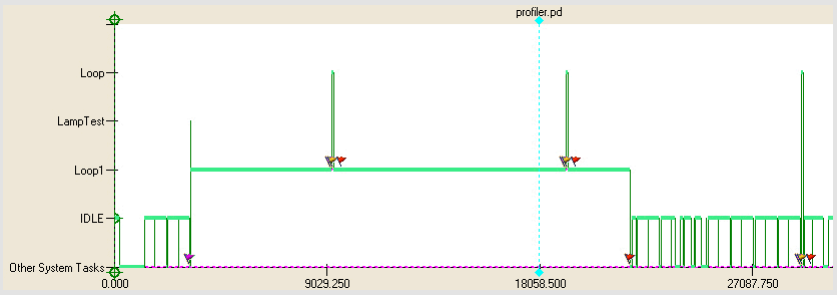


Figure 51: The "Loop" task interrupts the "Loop1" task - Display in the graphic view of the Profiler

When the "Loop1" task is executed in task class #4, the input image is available until the task has been completely executed.

5.1.4 Reading Profiler data after an error

The Profiler data can be uploaded by the target system at any time after stopping the recording. In case of an error, the Profiler data is also retained. It can then be determined which event, for example, led to the system being restarted.
The Profiler configuration should be adjusted so that the Profiler data is kept after restarting. The target memory for Profiler data and the Profiler configuration is set to "USERROM".

Diagnostics and service \ Diagnostics tools \ Profiler \ Preparing the Profiler \ General settings

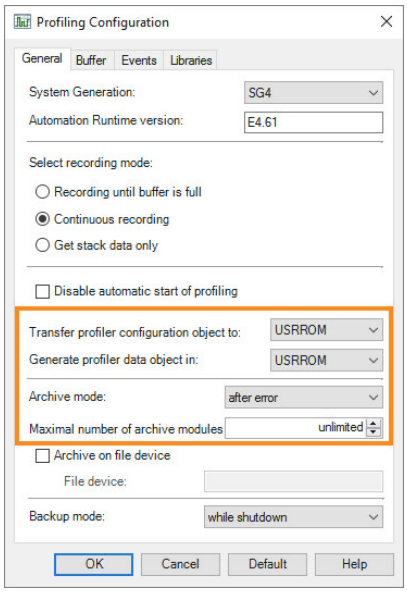


Figure 52: Settings for the target memory of Profiler data and the Profiler configuration

Exercise: Cause a cycle time violation and evaluate the Profiler data

A cycle time violation must be caused in the "Loop" task by increasing the variable "udEndValue".
After restarting the target system in the SERVICE operating mode, open the Profiler and load the Profiler data from the target system.

1) Set the value of the variable "udEndValue" in the variable monitor to 500000

2) After restarting in the "SERVICE" mode, open the Profiler in the menu under "Open" \ "Profiler."

What happens if a timeout occurs?
If the configured **cycle time + tolerance** was exceeded during runtime, Automation Runtime triggers an exception. If the application program is not configured to handle this exception, the target system will restart in the "SERVICE" operating mode.

To upload the Profiler data, click the "**Upload data object**" icon in the toolbar. If there is an error, a new Profiler file is generated upon restart, which is given a timestamp. The corresponding file can be selected from a list during the uploading process.

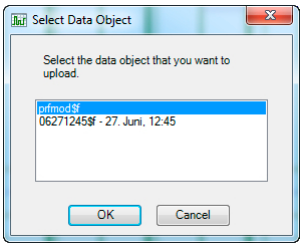


Figure 53: Selection of the profiler data

Profiler data can be filtered to limit the events being displayed. Which events should be displayed depends on the situation itself.

Diagnostics and service \ Diagnostics tools \ Profiler \ Recording Profiler data \ Analyzing Profiler data

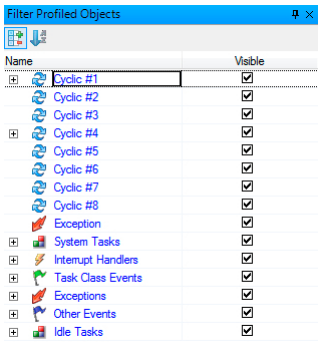


Figure 54: Filtering Profiler data

At a certain point in time, the time it takes to complete the task exceeds the configured cycle time and tolerance. This event (exception) is indicated by an appropriate icon.

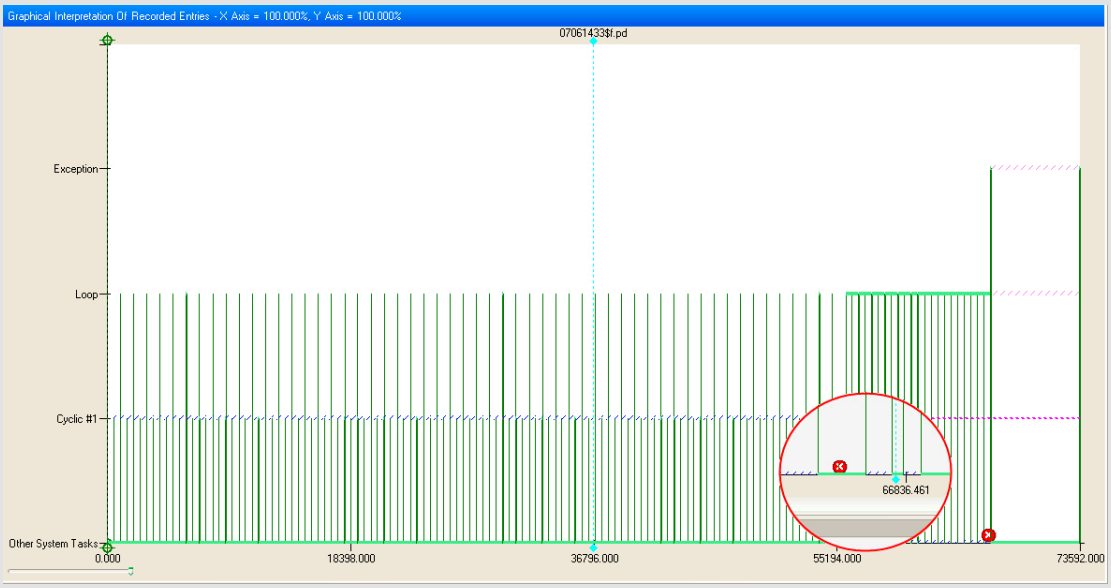


Figure 55: Exception in the profiler data

To analyze the cause, the data that comes before this point in time must be observed.
Using the measurement cursor and zooming in as necessary on the Profiler data are two ways that the data can be analyzed.

Execution time of the task "Loop"

The image shows that the task "Loop" is normally carried out in less µs (blue arrow). In the case of cycle time violation, the configured cycle time + tolerance is exceeded (red arrow).
This image shows how a simple application is recorded in the Profiler. The cause of a problem is generally harder to detect in real applications since there are usually several tasks / task classes running. By setting filters, specific processes can be targeted.

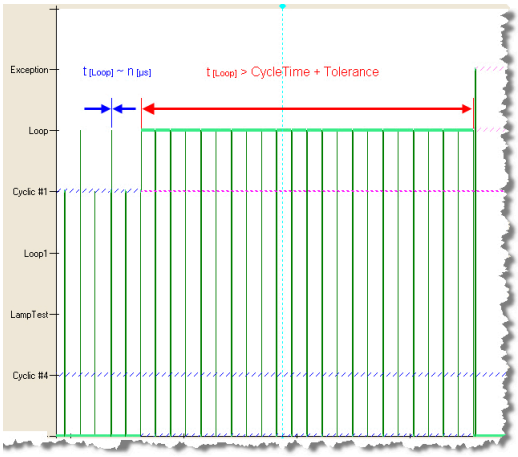


Figure 56: Determining the cycle time violation

Two tasks are running in task class #1 that usually finish executing within the configured task class cycle.

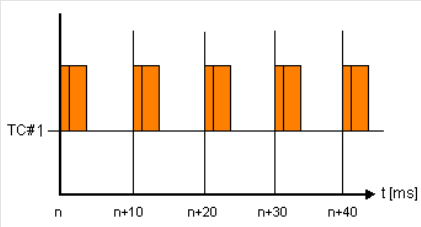


Figure 57: Execution times in task class #1

If it takes longer to complete the first task (beyond n+30 ms in the diagram) and the completion time for both tasks together exceeds the configured cycle time plus tolerance, then it will be the second task that is entered as the cause of the error although it is not really the main reason for the cycle time violation.

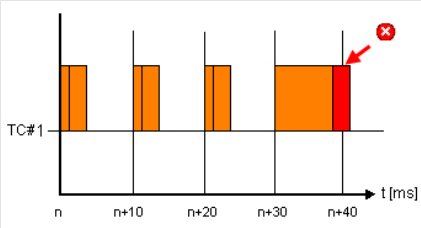


Figure 58: Execution times in task class #1 exceed configured cycle time

The sequence of events can be analyzed chronologically by evaluating the raw data ("**Output data**" icon in the toolbar).
The start and end of the call of the "Loop" task is entered in this list. If the time sequence is continued, it can be determined that the task has been started but has not ended.

Raw Data Of Recorded Entries

Nr.	Name	Event	Event Description
465	TickGen	0x000...	'TickGen' is now running - 'IOScheduler' was waiting
466	Cyclic #1	0x001...	'Cyclic #1' is now running - 'TickGen' was waiting
467	Cyclic #1	0x1e0...	Task class 'Cyclic #1' started
468	Cyclic #1	0x1e0...	Input scheduler of task class 'Cyclic #1' finished
469	Loop	0x020...	Cyclic task 'Loop' started
470	Loop	0x020...	Cyclic task 'Loop' finished
471	Cyclic #1	0x1e0...	End of cyclic programs in task class 'Cyclic #1'
472	Cyclic #1	0x1e0...	Output scheduler of task class 'Cyclic #1' started
473	DdXZXAcc.IF6	0x007f...	'DdXZXAcc.IF6' is now running - 'Cyclic #1' was waiting
474	tEpIV2If.IF3	0x007f...	'tEpIV2If.IF3' is now running - 'DdXZXAcc.IF6' was ready
475	DdXZXAcc.IF6	0x007f...	'DdXZXAcc.IF6' is now running - 'tEpIV2If.IF3' was delayed and waiting

Figure 59: Raw data for a Profiler recording

5.2 Searching for errors in the source code

When it comes to software, statistics have shown that there are usually around 2 to 3 errors contained in every 1000 lines of code.

Automation Studio's comprehensive diagnostics tools enable program errors and their causes to be found quickly.

5.2.1 Monitor mode in the program editor

The monitor mode is a handy feature that is very helpful when troubleshooting. Variables can be observed and analyzed in different ways.

The monitor mode is enabled and disabled using the "Monitor" icon in the toolbar. Alternatively **<CTRL>** can be pressed.

Figure 60: Enable monitor mode

Tooltips in source code

The value of the variable is displayed as a tooltip. The source code can be a textual or graphical programming language.

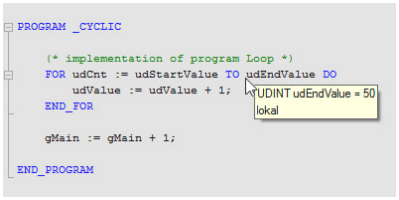


Figure 61: Tooltip in source code

Value display in visual programming languages

The value is shown right beside the variable name in visual programming languages.

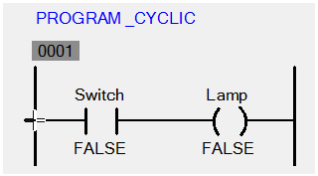


Figure 62: Visual programming languages in monitor mode

Watch window

If monitor mode is enabled, the variable monitor is displayed next to the source code. It is also possible to open the Watch window from a task's shortcut menu in the software configuration or from the online software comparison.

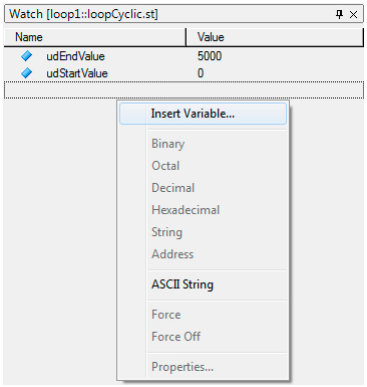


Figure 63: Watch window view

5.2.2 Powerflow

With the visual programming languages, the course of a signal can be displayed. The signal flow is enabled using the "Signal flow" icon when the monitor mode is active.

Figure 64: Display signal flow

When enabling the signal flow display, the message for disabling the cycle time monitoring must be confirmed.

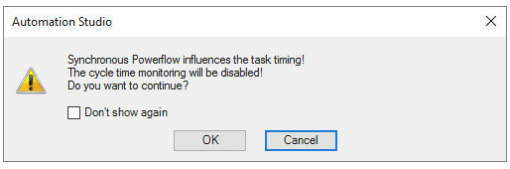


Figure 65: Message for deactivating the cycle time monitoring

Exercise: Powerflow in Ladder Diagram

The objective of this exercise is to enable powerflow in the "LampTest" program. The signal flow is changed in the variable monitor by writing the variable "Switch".

1) Open the "LampTest" program when the online connection is active
2) Enable monitor mode
3) Add the variables "Switch" and "Lamp" in the Watch window
4) Set variable "Switch"
5) Monitor results

If the contact condition for the "Switch" variable is fulfilled, the "Lamp" coil is set and the signal path is colored.

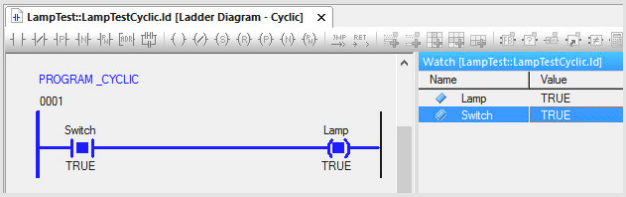


Figure 66: Powerflow enabled in Ladder Diagram

Diagnostics and Service \ Diagnostics tools \ Monitors mode \ Programming languages in monitor mode \ Powerflow

5.2.3 Line coverage

Line coverage is switched on and off with the "Line coverage" icon when the monitor mode is enabled.

Figure 67: Enable line coverage

When the line coverage is switched on, a message from Automation Studio appears. This message writes that cycle time monitoring is disabled when the line coverage is enabled. This message must be confirmed with "OK".

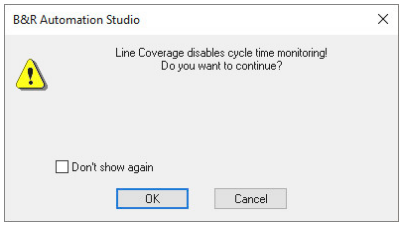


Figure 68: Message for deactivating the cycle time monitoring

Activated line coverage makes it possible to see exactly which lines are being run at which time. The active program lines are marked with a green symbol.

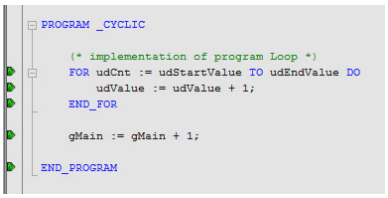


Figure 69: Active line coverage in Structured Text

Diagnostics and Service \ Diagnostics tools \ Monitors mode \ Programming languages in monitor mode \ Line coverage

5.2.4 Contextual Watch

Contextual watch is a tool for displaying the value of variables at defined source code positions. The real-time capability of the target system is not affected.
Contextual Watch combines the advantages of different Automation Studio diagnostics tools. It enables viewing the values of ANSI C and C++ symbols (local and global variables, parameters, etc.) as is possible in the debugger; but the runtime system does not have to be stopped for this. As with Line Coverage, Contextual Watch can be used to determine that specific source code lines are being executed, as well as the values of symbols at that exact position. In contrast to PV Watch, Contextual Watch provides certainty as to when the value was determined during program execution.

Contextual Watch is not supported on simulated target systems (ARsim).

The Contextual Watch is enabled using the "Contextual Watch" icon when the monitor mode is enabled. The corresponding display of the Contextual Watch is part of the output window.

Figure 70: Enabling Contextual Watch

When Contextual Watch is enabled, a message appears, which must be confirmed. It should be noted that the values in Tooltips are not synchronized with the values in the Contextual Watch. Furthermore, Contextual Watch requires an additional runtime for each task class. This is important in order to ensure that there are no cycle time violations during the diagnosis with Contextual Watch.

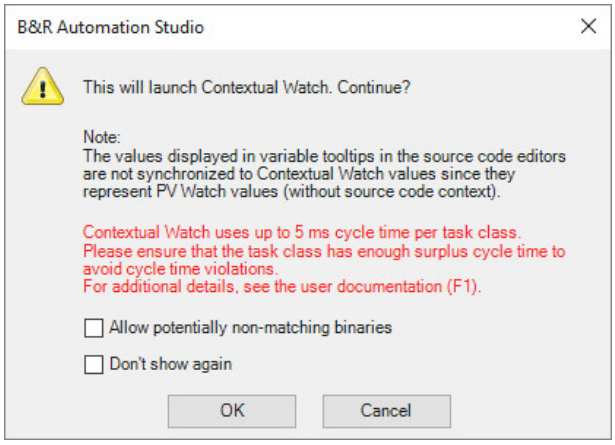


Figure 71: Message when activating Contextual Watch

In this example, the variable "udValue" is used in several places in the program. Contextual Watch is activated and the variable is dragged and dropped into the output window. It can be seen that the variable value is different depending on the source code position.

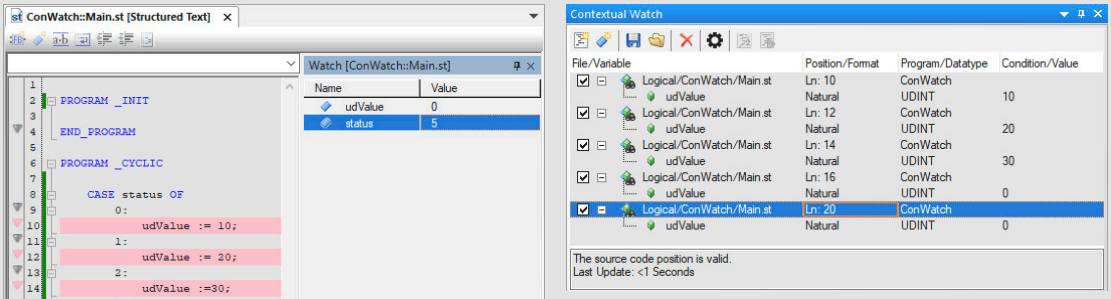


Figure 72: Source code with variable "udValue"; color-marked source code lines were inserted into Contextual Watch

Figure 73: Representation of the "udValue" values in Contextual Watch

Exercise: Observe program "Loop" in Contextual Watch

In the Watch window of the "Loop" program, it was observed that the value of the variable was displayed at the end of the program. The objective of this task is to display variables that are used several times with Contextual Watch and their values in a context-dependent manner.

1) Program "Loop" at the end of the program to expand the following line

```
udCnt;
```

2) Transfer the program to the target system

3) Enable monitor mode and Contextual Watch

4) Add the "udCnt" variable to the Contextual Watch

5) Check results

Diagnostics and service \ Diagnostics tools \ Contextual Watch

5.2.5 Debugging the source code

An important tool in Automation Studio is the debugger. Troubleshooting the source code or a library is made easier for programmers.

Debugging possibilities in Automation Studio

- Line by line execution of the program and parallel variable monitoring in the automatic Watch window
- Stopping the application using breakpoints
- Stepping into called functions, e.g. in library functions / function blocks as long as the source code is available.

Exercise: Find errors in a Structured Text program using the debugger

Create a Structured Text program called "dbgTest".
Add a USINT array called "AlarmBuffer" with a length of 10 and a UINT variable called "index" to the "dbgTest.var" file.

In the cyclic part of the program, the array initializes with any value, e.g. 112.
The following – faulty – program code contains one of the most commonly made errors.

Program code

```
PROGRAM _CYCLIC
    FOR index :=0 TO 10 DO
        AlarmBuffer[index] := 112;
    END_FOR
END_PROGRAM
```

Table 7: Faulty program subroutine

When the program is started, the value 112 is written to each of the 10 elements of the array. The program seems to be working.
Error overview: There is write access to index 0 to 10. The array only has 10 elements (index 0 to 9). This type of error is often difficult to detect at first glance and causes the program to overwrite the following memory locations.

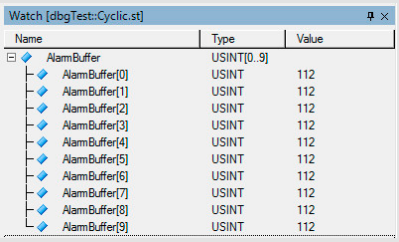


Figure 74: Watch window in monitor mode

The error overview is analyzed using the debugger and the Watch window.

Prepare debugging

Step 1

Switch on the monitor mode using the "Monitor mode" icon.

Figure 75: Enable monitor mode

Step 2

Enable the debugger using the "Debugger" icon

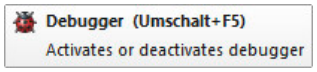


Figure 76: Enable the debugger.

Step 3

Add the "AlarmBuffer" variable to the Watch window.

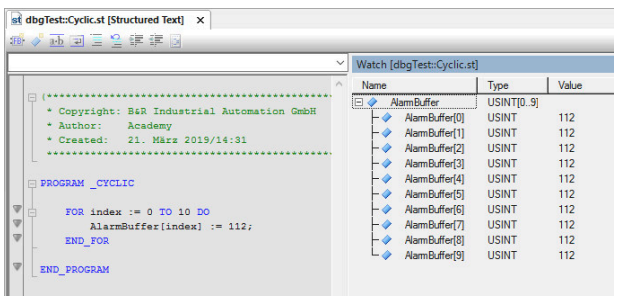


Figure 77: Monitor mode in the program editor, left program code, right Watch window

Step 4

Place the cursor in the first line of the FOR loop. Add a breakpoint using the "Set breakpoint" icon.

Figure 78: Set the breakpoint

Reaching a breakpoint stops the entire application running on the target system.

"Step into" debugging

Step 1	Step 2
First, the elements of the "AlarmBuffer" array are manually changed to the value 0 in the Watch window.	Execute the single steps of the program code using the "Single step" icon.  Figure 79: Execute the single steps

If the debugger hits a breakpoint, then the active line is indicated by a yellow marker.

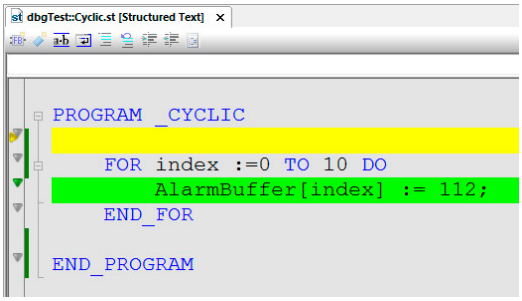


Figure 80: Active step marked yellow

Each time you press the **<F11 key>** the loop is traversed and an array element is defined. The single steps are executed until all array elements are defined with the assigned value.

Name	Type	Value
AlarmBuffer	USINT[0..9]	
AlarmBuffer[0]	USINT	112
AlarmBuffer[1]	USINT	112
AlarmBuffer[2]	USINT	112
AlarmBuffer[3]	USINT	112
AlarmBuffer[4]	USINT	112
AlarmBuffer[5]	USINT	0
AlarmBuffer[6]	USINT	0
AlarmBuffer[7]	USINT	0
AlarmBuffer[8]	USINT	0
AlarmBuffer[9]	USINT	0

Figure 81: Step-by-step writing to the variables

The last array element with the index 9 also receives the assigned value.
If the single steps are continued, the loop is executed again. The index has the value 10. This memory area is outside the array and can cause problems.

This type of error can be detected by the IEC Check library, see 5.2.6 "IEC Check library" on page 40.

Diagnostics and service \ Diagnostics tool \ Debugger

5.2.6 IEC Check library

The IEC Check library contains functions for checking division operations, range violations, proper array access as well as reading from or writing to memory locations.
The corresponding checking function is called by the program (supported IEC 61131-3 languages or Automation Basic) before each of these operations is carried out.
With the IEC Check library, the user can use a dynamic variable to determine what should happen in the event of a division by zero, an out of range error or an illegal memory access.

Programming \ Libraries \ IEC Check library

5.2.7 Evaluating event IDs, status variables and return values

Return values and status values of functions and function blocks must be evaluated in the user program. The terms "status" and "event IDs" should be understood as synonyms in this context.

The following example shows a function block being called. This function returns a status that can be used to determine whether an error has occurred during the call. A list of the status values or the event IDs are documented in the description of the library used.

```
2:  (*Read information from an AR logger user module*)
    Logger.AsARLogGetInfo_0.enable := 1;
    Logger.AsARLogGetInfo_0.pName :=  ADR('usrlog');
    Logger.AsARLogGetInfo_0;  (*Call the Functionblock*)

    (*AsArLogGetInfo successful*)
    IF Logger.AsARLogGetInfo_0.status = 0 THEN
        Logger.Step := 0;
    ELSIF Logger.AsARLogGetInfo_0.status = ERR_FUB_BUSY THEN
        (*Busy*)
    ELSE  (*Go to Error Step*)
        Logger.Step := 10;
    END_IF
```

Figure 82: Status evaluation of function blocks

5.3 Using variables in the programs

The proper usage of variables in the different programs that are in the Logical View can be checked by creating a cross-reference list or explicitly searching for a known variable name.

5.3.1 Generating cross-reference list

The cross-reference list indicates which process variables, functions and function blocks can be used at which point in the project.
The cross-reference list is optional and can be generated when the project is compiled (built); the results are then displayed in the output window under the "Cross Reference" tab.
To generate a cross-reference list, you must activate it using the following menu option: "Project" / "Settings". Alternatively, the cross-reference list is generated using the menu option "Project" / "Build cross-reference".
Frequent search tasks can be handled easily with the help of the cross reference list.

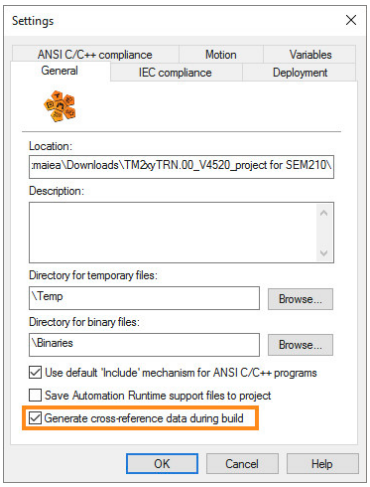


Figure 83: Enabling the creation of cross-reference list during build

Exercise: Generate a cross-reference list

Create a cross-reference list for the open project.

1) Enable the cross-reference list in the project settings
2) Compiling a project
3) Check the cross-reference list in the output window

You can analyze the cross-references for variables and their attributes in the output window.
If a variable is selected on the left side, its usage and the type of access will be displayed in the source code or in the I/O allocation on the right side.

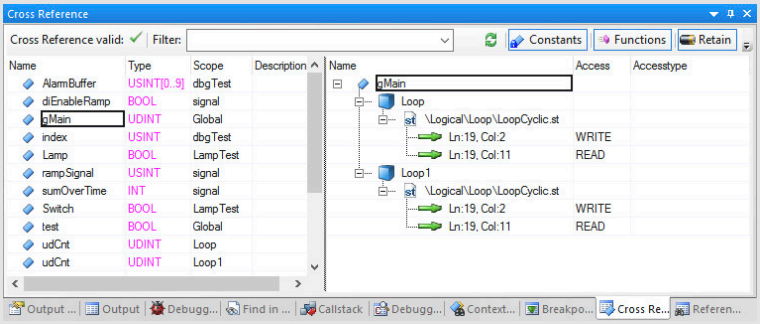


Figure 84: Display in the cross reference list

Project management \ Workspace

- Main menu \ Project
- Output window \ Cross-reference
- General project settings

5.3.2 Searching in files

If you are looking for matches of names, product IDs or comments, you can search in the project files.
To search for a variable, use "Edit" / "Find and Replace – Find in Files" or press **<CTRL> + <Shift> + <F>**.

The search term is entered in the dialog box. The result of the search is displayed in the output window in the "Find in Files" tab.

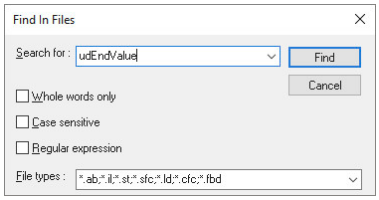


Figure 85: Searching in files

Double-clicking on a result in the output window opens the respective source file and places the cursor at the corresponding position.

Project management \ Workspace \ Find in files

5.4 Source file comparison

With the Automation Studio source file comparison, it is possible to compare individual files, folder elements (programs, libraries, packages and data objects) or entire projects.
During local source file comparison, an element from the currently open project or the entire project is always compared with another element or project on a data storage device.

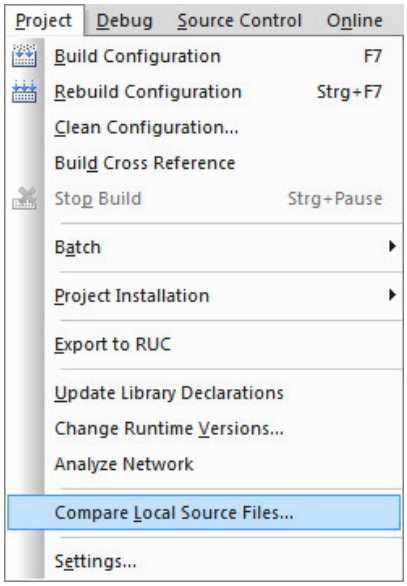


Figure 86: Opening the source file comparison

Overview comparison

After opening the source file comparison, an overview comparison is performed first of all. The structure of both projects is compared. For example, only different objects are displayed via filters.

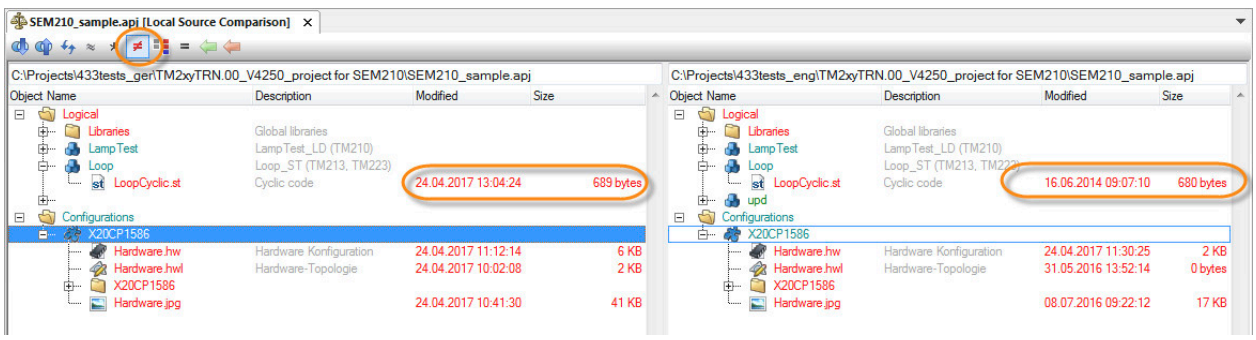


Figure 87: Overview comparison with a filter - only the differences are displayed.

Detailed comparison

By double-clicking on an object, both source files are opened in a detailed comparison. The editor highlights the differences in the corresponding text-based or tabular representation.

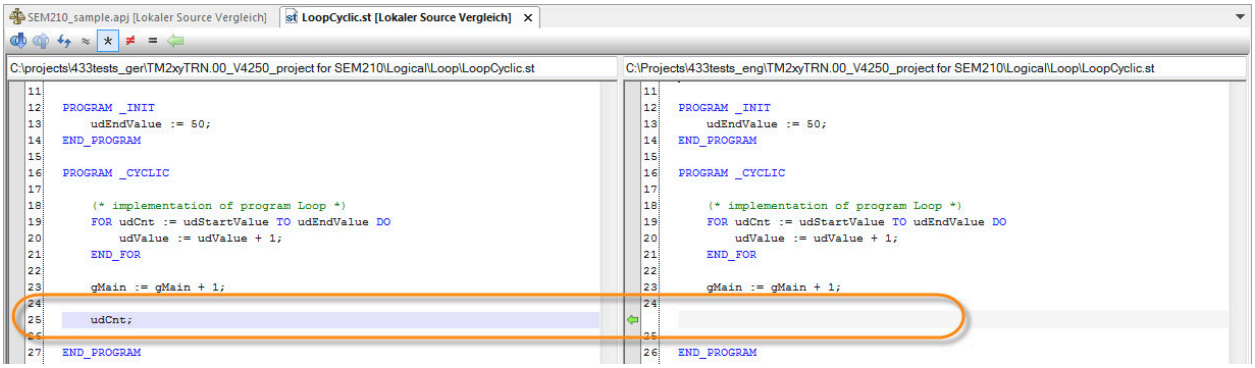


Figure 88: Structured text program in detailed comparison

Project management \ Automation Studio source file comparison

- Overview comparison
- Detailed comparison

5.5 Source file comparison on the target system

The source file comparison compares the source files of the local project with those of the target system. The requirement for this is that the source file comparison is enabled on the target system before the last transfer of the configuration.

Step 1

Enable the source file comparison on the target system using the shortcut menu "Properties".

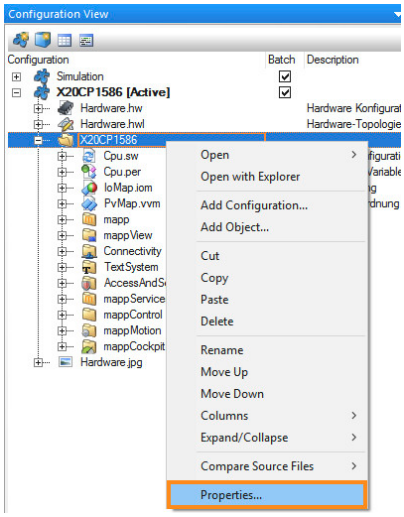


Figure 89: Shortcut menu in the Configuration View

Step 2

Activate the checkbox "Enable source file comparison on target system" in the "Comparison" tab.

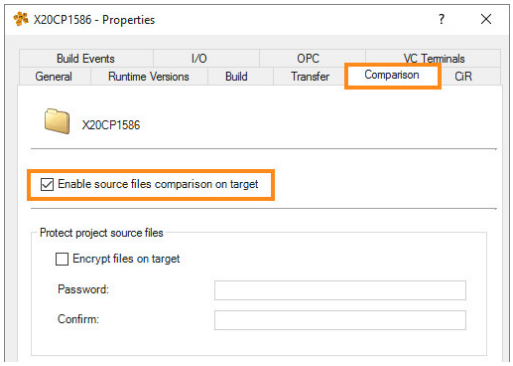


Figure 90: Enable the source file comparison in the target system

Step 3

Transfer configuration to the target system.
After transferring the project to the controller, source file comparison is available for the target system.

Step 4

Open the source file comparison using the shortcut menu of the source file "Compare source file" \ "On the target system".

Step 3

Step 4

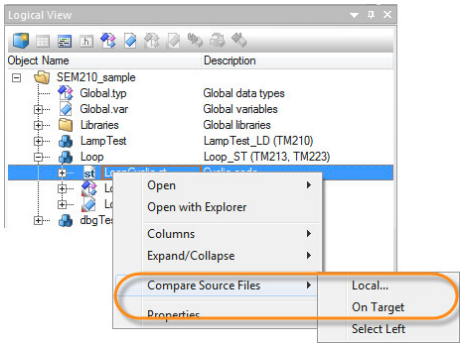


Figure 91: Open source file comparison on target system

The source files on the target system and in the project are displayed parallel to each other. The differences are highlighted in color and can be filtered in the toolbar. Changes in the source code can be transferred to the project using the toolbar.

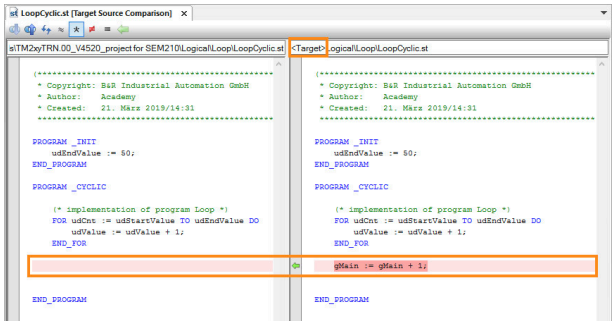


Figure 92: Displaying source files in the project and on the target system

Project management \ Automation Studio source code file comparison \ Source code file comparison on the target system

6 Making preparations for servicing

It is necessary during the configuration, commissioning and testing of the application to prepare the machine or system for service activities that may occur later.

6.1 System Diagnostics Manager (SDM)

The System Diagnostics Manager (SDM) can be used to diagnose the controller via a web browser[2].
The only requirement for these diagnostics is an Ethernet connection to the controller.

SDM functions:

- General system overview
- Showing and saving Logger files
- Overview of installed software objects
- Hardware modules and I/O status
- Motion control diagnostics
- Creating system dumps

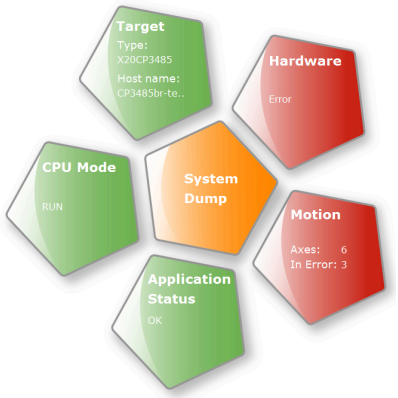


Figure 93: SDM startup screen

Diagnostics and service \ Diagnostics tools \ System Diagnostics Manager (SDM)

6.1.1 Enabling SDM

SDM is automatically enabled when creating a new project or new configuration.
The SDM configuration is opened in Physical View using the **"Configuration"** option in the controller's shortcut menu.

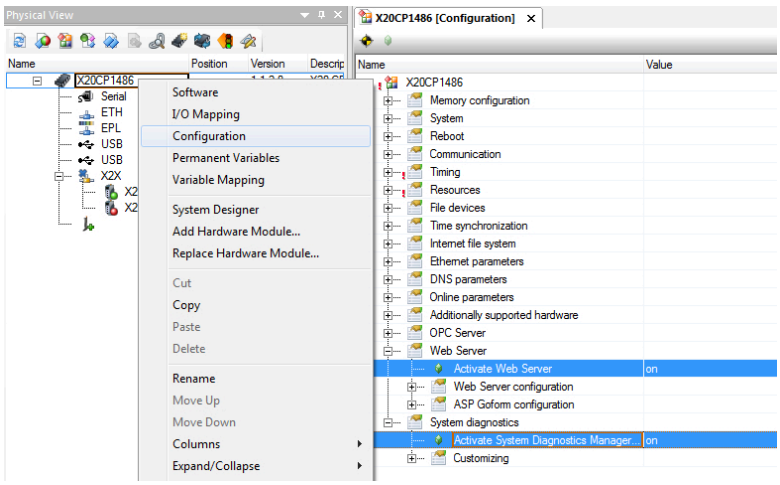


Figure 94: Opening the configuration, SDM and web server settings

Are there requirements for using the SDM?
System Diagnostics Manager requires the web server service, which is also a component of Automation Runtime.

2 B&R recommends using Google Chrome as the browser.

Exercise: Check the System Diagnostics Manager configuration

Check whether the web server and System Diagnostics Manager are enabled in the Automation Runtime configuration.

1) Open the Automation Runtime configuration

2) Enable the Web Server and System Diagnostics components

6.1.2 Accessing the SDM

Before connecting to the controller, the network configuration of the PC in use may have to be modified. It is recommended to make a note of the original settings before doing this.
A standard network cable is used to connect with the controller. The LED status indicators of the PC and the controller indicate whether there is an online connection or not.
The connection to SDM is made via the controller's IP address or hostname. SDM is accessed via the browser[3] using link**"http://ip-address/SDM**".

Step 1 - Connect PC to controller

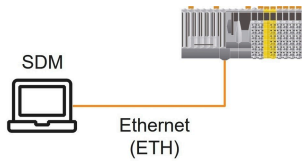


Figure 95: Connecting PC to controller via network cable

Step 2 - View SDM in web browser

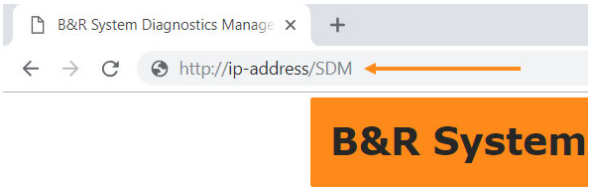


Figure 96: Accessing the SDM via a web browser

Which browser is required?
An SVG-capable web browser is required to view SDM pages. Most current web browsers support this. An SVG plug-in is offered when using older browsers. If viewing SVGs is still not possible, then an HTML view of System Diagnostics Manager can also be accessed via "**http://IP-address/sdm-vga**".

Which IP address must be entered in the browser?
The controller's IP address or hostname can be found in the documentation provided or requested directly from the manufacturer. In some cases, the set IP address is written on the control cabinet.

When can the SDM be opened?
The target system IP address is checked in the controller's Ethernet configuration. If there is already an active connection to the controller, SDM can be opened using the **"Extras" / "System Diagnostics Manager"** menu option.

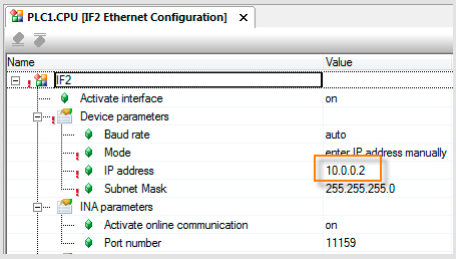


Figure 97: Check the network settings in the controller's Ethernet configuration

Diagnostics and service \ Diagnostics tools \ System Diagnostics Manager (SDM)

3 B&R recommends using Google Chrome as your browser.

Exercise: Access SDM using a web browser

Enter the URL for accessing the SDM, e.g. http://10.0.0.2/SDM

1) Open the web browser
2) Enter the URL for accessing the SDM

Exercise: Evaluate Logger with SDM and in Automation Studio

Assumption: The system has booted into the SERVICE operating mode for no apparent reason. Unfortunately, Automation Studio is not available on site. The Logger file "**$arlogsys**" should be read out using the SDM and then opened in the Automation Studio Logger.

1) Establish a connection to the SDM and change to the "Logger" page
2) Open the Logger file "$arlogsys".
3) Save Logger file with extension ".br"
4) Opening a Logger File in Automation Studio
5) Load Logger file with extension ".br"
6) Select the error entry and press the **<F1 key>** to obtain detailed information

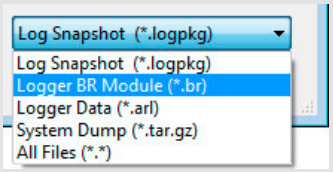


Figure 98: Select the file type with the .br file extension in Automation Studio

If a system dump was generated in the System Diagnostics Manager with the option "Parameters + Data files", then it can be opened and displayed directly with the Automation Studio Logger.

Entries are shown in the SDM Logger without additional supplementary information. This can only be displayed in Automation Studio.

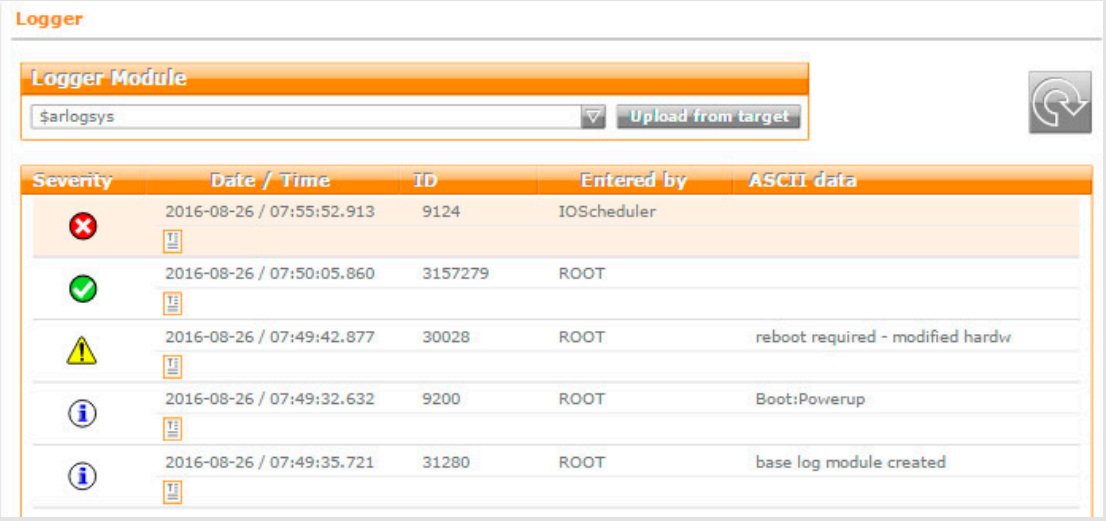


Figure 99: Logger entries in System Diagnostics Manager

The online data must be deselected in the Automation Studio Logger; otherwise, the online entries are displayed with the entries loaded from the file.

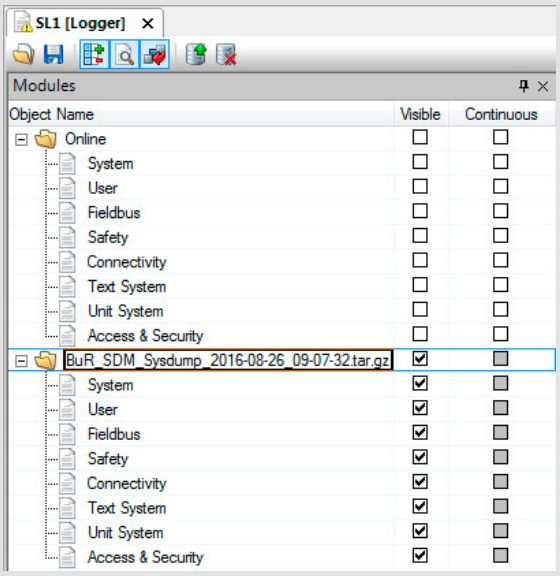


Figure 100: Disabling the online Logger entries

6.2 Query the status of the battery

Depending on the used target system, a battery is used for data buffering. Further details about this are available in the data sheet of the used controller. The backup battery for the real-time clock and the nonvolatile variables (retain, permanent) being used in the application can be monitored from the application itself.

Figure 101: "4A0006.00-000" lithium battery

When must the battery be changed?
Note the replacement of the battery in the service instructions for the machine / system. The life of the battery is specified in the documentation for the controller being used.

The battery status can be queried in a number of ways:

- AsHW library in the application
- Control I/O mapping
- Online info dialog box
- System Diagnostics Manager

The I/O mapping is opened using the **"I/O mapping"** option in the Physical View shortcut menu for the controller. The variable attached to the "BatteryStatusCPU" can be evaluated in the application program as needed.

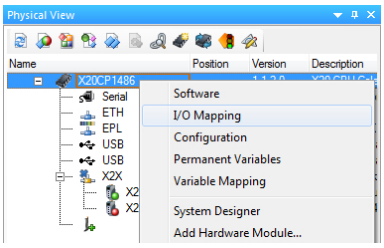


Figure 102: Opening the controller's I/O mapping

The values of the status data points in the I/O mapping of the controller can be checked in monitor mode, see 4.3 "I/O monitor and forcing" on page 24.

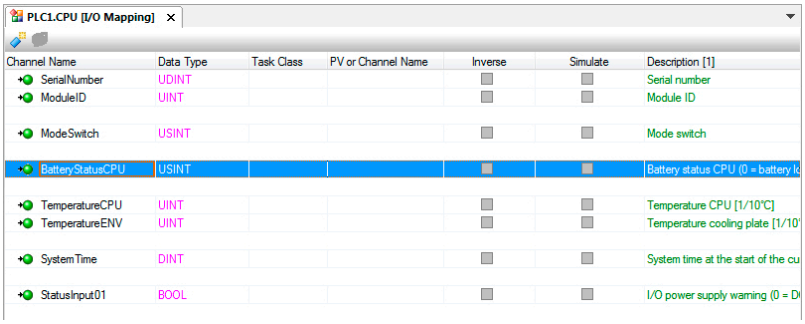


Figure 103: Querying the battery status in the controller's I/O mapping.

6.3 Runtime Utility Center service tool

The Runtime Utility Center is a system tool that provides a range of utilities for diagnostics and service on B&R controllers. The installation program for the Runtime Utility Center is included in the Automation Studio installation or can be downloaded separately from the B&R website.

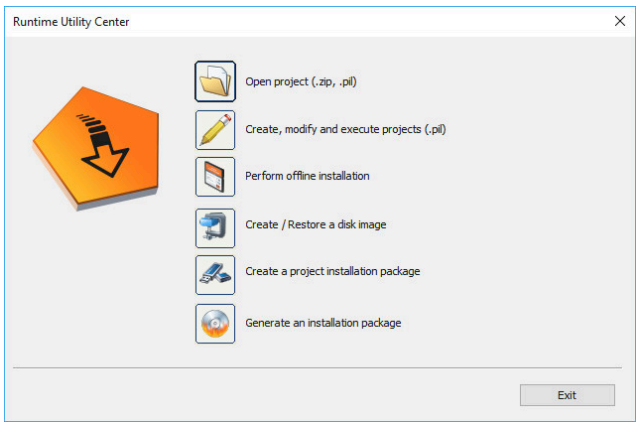


Figure 104: Runtime Utility Center start page

The most important functions are:

- Performing service functions via an online connection to the controller
- Variable functions for backing up and restoring process variables
- Creating individual Instruction Lists for testing and installation procedures
- Backing up and restoring a CompactFlash/CFast card
- Offline installation of a control project on a CompactFlash/CFast card
- Creating project installation packages for USB installation
- Custom mode allows the creation of a user-defined user interface

How can I open the Runtime Utility Center help documentation?
The Runtime Utility Center contains complete help documentation. This help documentation is opened by pressing the **<F1 key>** . The Runtime Utility Center must be opened before doing this. The following entries provide additional important information about using the Runtime Utility Center.

Runtime Utility Center \ Start page
Runtime Utility Center \ Operation \ Workspace
Runtime Utility Center \ Operation \ Commands \ Establish connection, wait for new connection
Runtime Utility Center \ Operation \ Commands \ PLC Info \ Logger

Downloading the Runtime Utility Center

The Runtime Utility Center is part of the **PVI development setup** and can be downloaded from the B&R website: www.br-automation.com → Downloads → "PVI Development Setup".

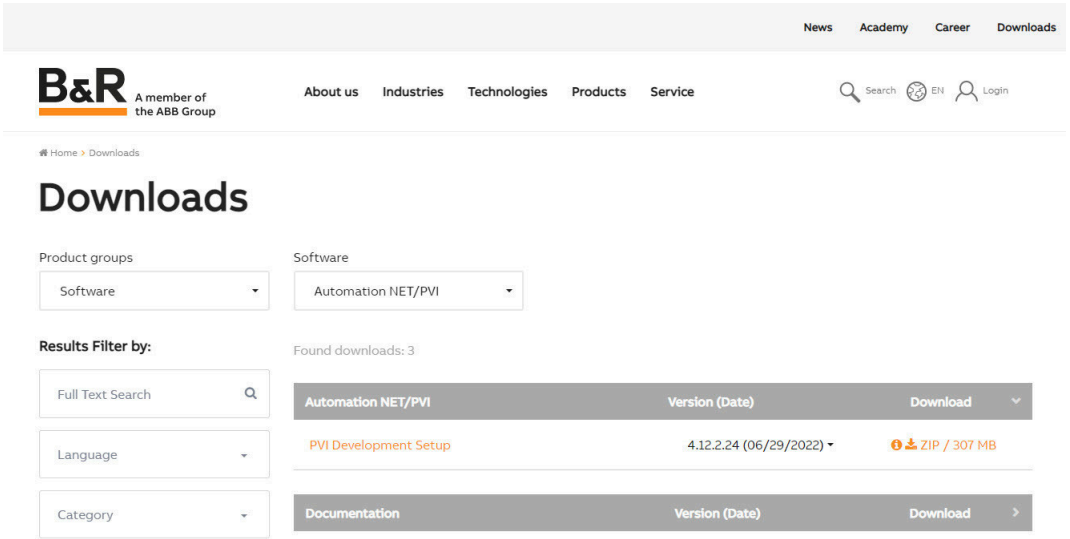


Figure 105: Downloads section, product group "Software" → "Automation NET/PVI"

Installing the Runtime Utility Center

The downloaded installation package must be extracted before installation. The installation program can then be started. No changes have to be made during the installation for use of the Runtime Utility Center.

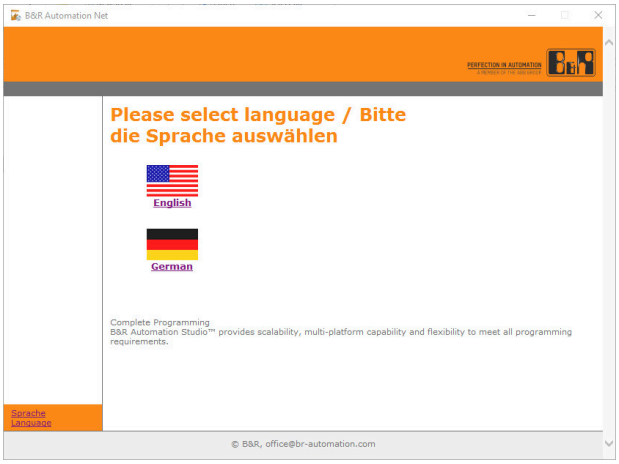


Figure 106: Select a language

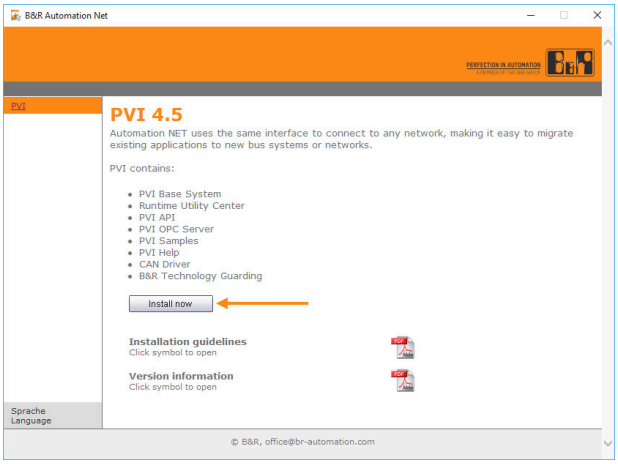


Figure 107: Clicking on "Start installation"

6.3.1 Runtime Utility Center package

Create Runtime Utility Center export

The Runtime Utility Center export is started from the Project menu in Automation Studio. After the destination folder is selected and confirmed, the necessary data is exported as a ZIP file.
The export file can then be processed with the Runtime Utility Center.

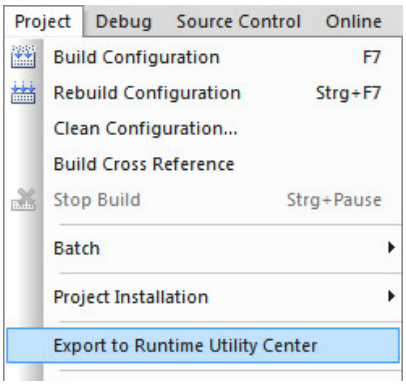


Figure 108: Runtime Utility Center (RUC) export in Automation Studio

Project management \ Project installation \ Performing project installation \ Export RUC

Loading Runtime Utility Center export package

Select "**Open project (.zip, .pil)**" to load the Runtime Utility Center export package. Then, the following functions are available:

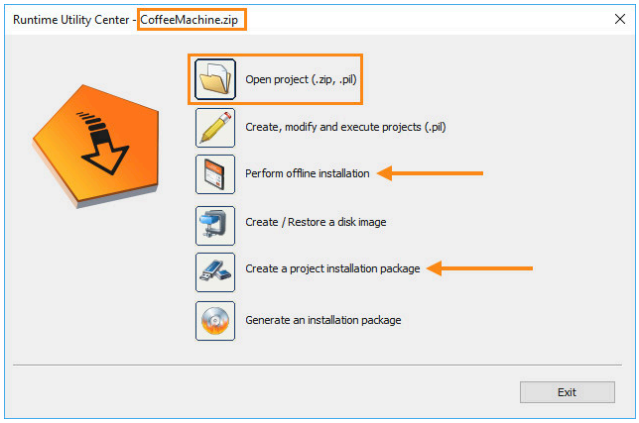


Figure 109: Runtime Utility Center start page with export file already loaded, see header of window

- Performing offline installation
  This function can be used to perform an initial transfer to a CompactFlash/CFast card.
- Creating project installation package
  This function can be used to create a project installation package, e.g. for USB installation.

Runtime Utility Center \ Creating a list / data medium \ Project installation

Exercise: Reinstalling the controller using a USB flash drive

The controller should be reinstalled without using Automation Studio. After a build of the project, a Runtime Utility Center export is created in Automation Studio.
The export file is opened in the Runtime Utility Center. A project installation package is then created for the USB flash drive.

1) Create Runtime Utility Center export
2) Start Runtime Utility Center
3) Open export file (*.zip)
4) Generating a project installation package

6.3.2 Backing up and restoring variable values

One function of Runtime Utility Center is to load variable values from the controller and to restore them at a later point in time.

Exercise: Save variable values

Due to mechanical damage to the system, the CPU must be replaced. To prevent recipe variables and other process data from being lost, the necessary information on the CPU is uploaded using Runtime Utility Center and then transferred later to the new CPU.
Create a Runtime Utility Center list that saves the values of variables in the "Loop" task.

1) Open the Runtime Utility Center from Automation Studio
2) Click on the menu option "Create, edit and execute projects (.pil)". A new project is created
3) Add the "Connection" command ("Command" \ "Connection") and enter the target system's IP address

4) Add the "Variable list" command ("Command" \ "Process variable function" \ "Variable list")

   Only the variables in the "Loop" task are being backed up in this example. The list can be stored to any directory

5) Start the Instruction List with the **<F5 key>**

Diagnostics and service \ Service tools \ Runtime Utility Center \ Operation

- Menus \ Start
- Commands \ Establish connection, wait for new connection
- Commands \ List functions

The variable values from the "Loop" task are backed up in the specified file.

Exercise: Restore variable values

The variable values backed up in the last task now need to be transferred to the controller using Runtime Utility Center. A Runtime Utility Center list is to be created that restores the values of variables.

1) Open the Runtime Utility Center from Automation Studio and create a new list via the menu ("File" \ "New")

2) Add the "Connection" command ("Command" \ "Connection") and enter the target system's IP address

3) Add the "Variable list" command ("Command") \ "Process variable function" \ "Transfer variable list to PLC")

   The variable list saved in the last task can be selected in the "Browse" dialog box.

4) Start the Instruction List with the **<F5 key>**

The variable values saved in the file are written to the corresponding variables in the "Loop" task.

If every variable from every task is backed up and then transferred back to the controller, be aware that writing to variables sequentially can cause unexpected behavior in the process.
If this situation is still necessary, it is recommended to put the controller in the SERVICE operating mode first.

7 Summary

Automation Studio offers several different tools for localizing problems and errors.
They need to be used sensibly in combination with analytical thinking. To be able to use these diagnostics tools effectively, it is necessary to get an overview of the situation, clarify the general conditions and examine these conditions from a certain distance.

Figure 110: Diagnostics

Only then can the circumstances be cleared up and analyzed in detail. A comprehensive overview of potential errors can be achieved by excluding and reducing the number of possible error sources, making it considerably easier to correct any errors that may still occur.

Automation Academy

Your knowledge advantage

The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. Expand your skills in the field of automation technology and learn to independently implement **efficient automation solutions** using B&R systems.

Decide for yourself which **learning concept** you prefer!

Classroom Learning

Virtual Classroom Learning

Online courses

Classroom learning

B&R offers **standard seminars** at all B&R locations. Services include seminar documents, effective communication of training course content by experienced trainers and an Automation Diploma. A combination of group work and self-study provides the high level of flexibility needed to maximize the learning experience.

Virtual classroom

Remote Lectures supplement B&R's continuing education portfolio with a virtual classroom, offering an alternative to our on-site seminars. Selected content from our standard seminars is offered online. In addition to remote learning methods, powerful simulation tools and secure remote maintenance are used.

Online courses

Take control of the content and learn at your own pace. With B&R **online courses**, you can take your first steps in the world of B&R automation at any time. Based on a comprehensive narrative, you will independently work out how to use our products. The mix of different media allows a logical sequence to be followed when learning as well as a targeted choice of information to be used as a reference.

Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.
Access additional information here:
https://www.br-automation.com/de/academy/

Enjoy your next training course!

AutomationAcademy

B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

TM223TRE.462-ENG